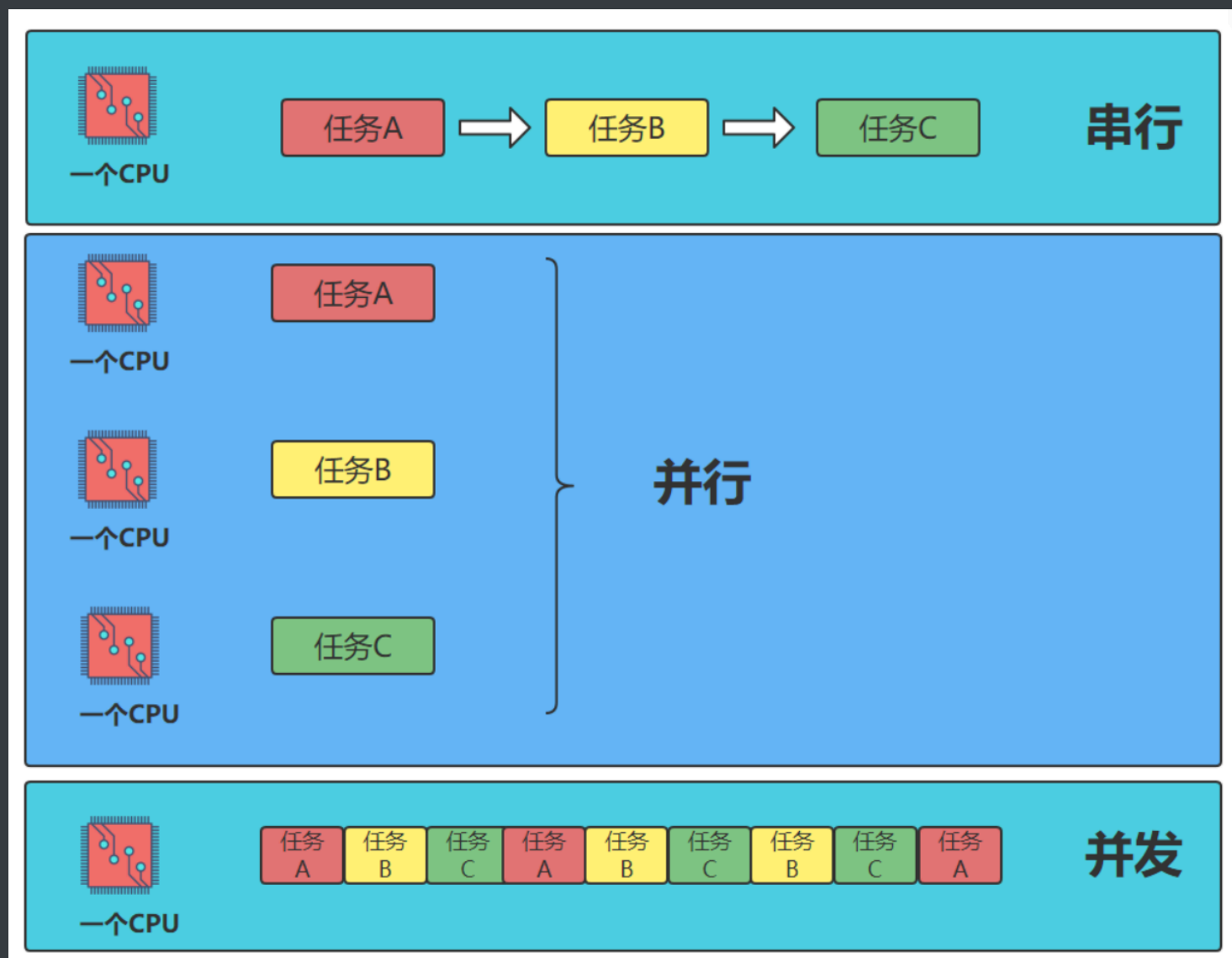


03 并发与网络编程

并发编程介绍

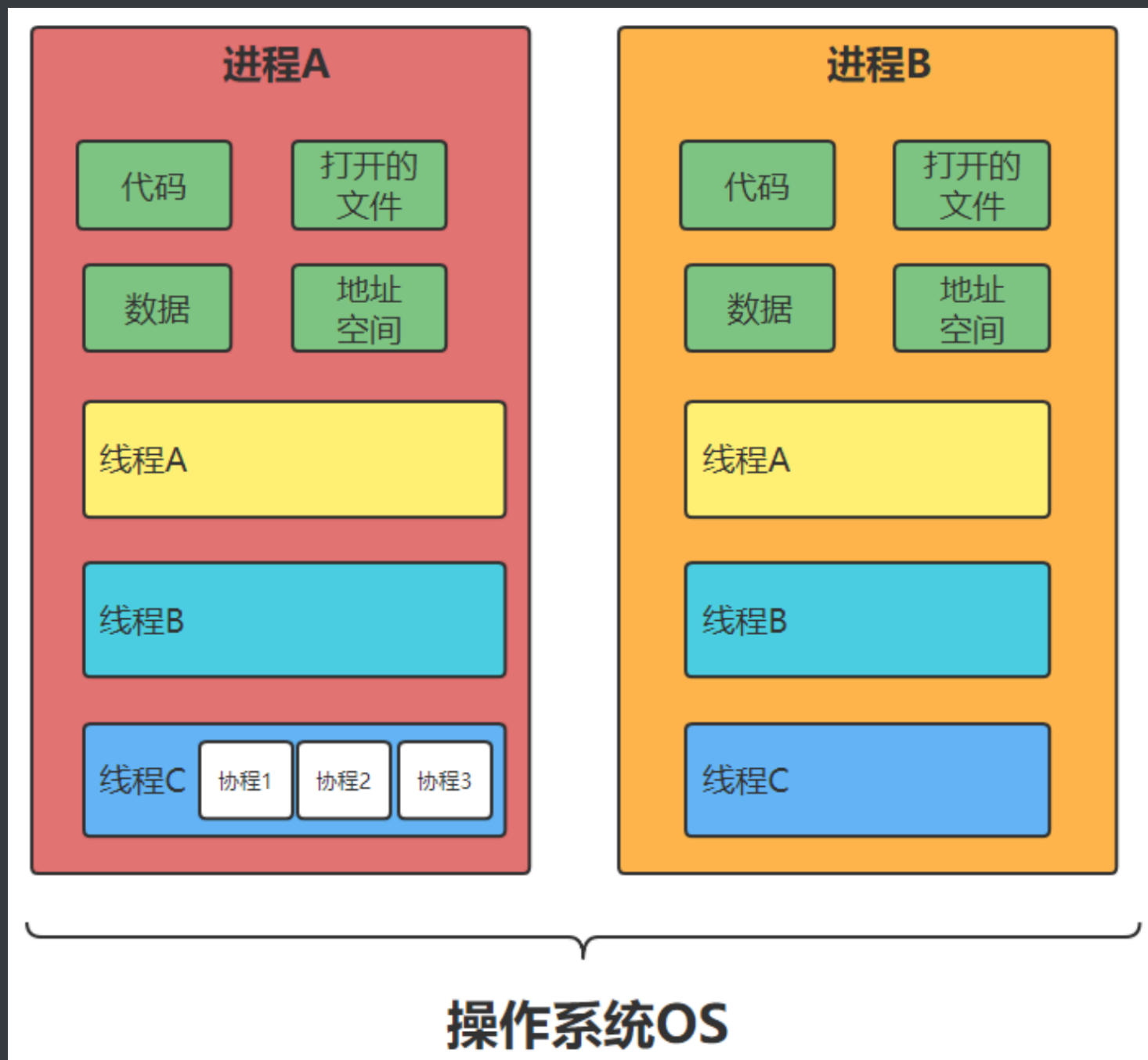
串行、并行和并发的区别



- 串行(serial): 一个CPU上，按顺序完成多个任务
- 并行(parallelism): 指的是任务数小于等于cpu核数，即任务真的是一起执行的

- 并发(concurrency): 一个CPU采用时间片管理方式, 交替的处理多个任务。一般是是任务数多于 cpu核数, 通过操作系统的各种任务调度算法, 实现用多个任务“一起”执行 (实际上总有一些任务不在执行, 因为切换任务的速度相当快, 看上去一起执行而已)

进程、线程、协程的区别



1. 线程是程序执行的最小单位; 晋城市操作系统分配资源的最小单位。
2. 一个进程由一个或者多个线程组成; 线程是一个进程中代码的不同执行路线。

3. 进程之间相互独立，但同一进程下的各个线程之间共享程序的内存空间（包括代码段、数据集、堆等）及一些进程级的资源（如打开文件和信号），某进程内的线程其它进程不可见。
4. 调度和切换：线程上下文切换比进程上下文切换要快的多。

- 进程(Process)：拥有自己独立的堆和栈，既不共享堆，也不共享栈，进程由操作系统调度；进程切换需要的资源很最大，效率低
- 线程(Thread)：拥有自己独立的栈和共享的堆，共享堆，不共享栈，标准线程由操作系统调度；线程切换需要的资源一般，效率一般（当然了在不考虑GIL的情况下）
- 协程(coroutine)：拥有自己独立的栈和共享的堆，共享堆，不共享栈，协程由程序员在协程的代码里显示调度；协程切换任务资源很小，效率高

同步和异步通信机制的区别

同步和异步强调的是消息通信机制 (synchronous communication/ asynchronous communication)。

同步(synchronous)：A调用B，等待B返回结果后，A继续执行

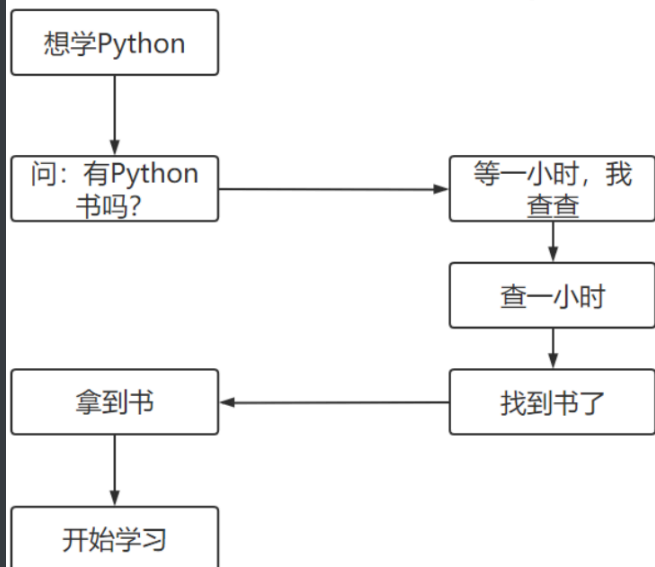
异步(asynchronous)：A调用B，A继续执行，不等待B返回结果；B 有结果了，通知A，A再做处理。



老高



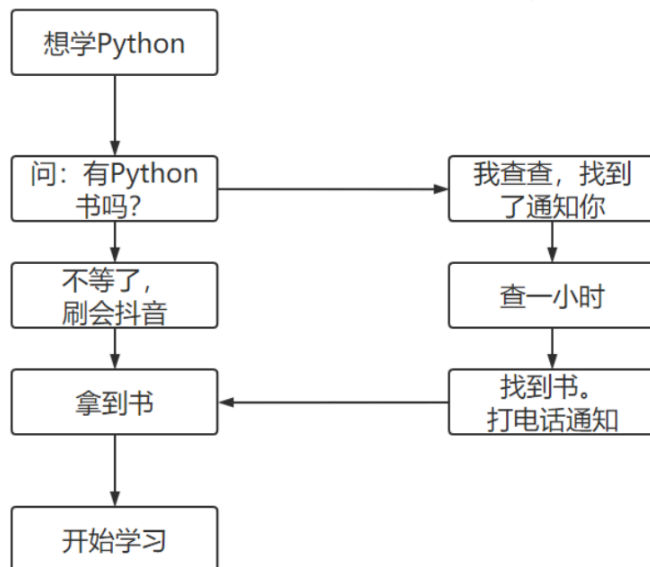
同步书店
老板



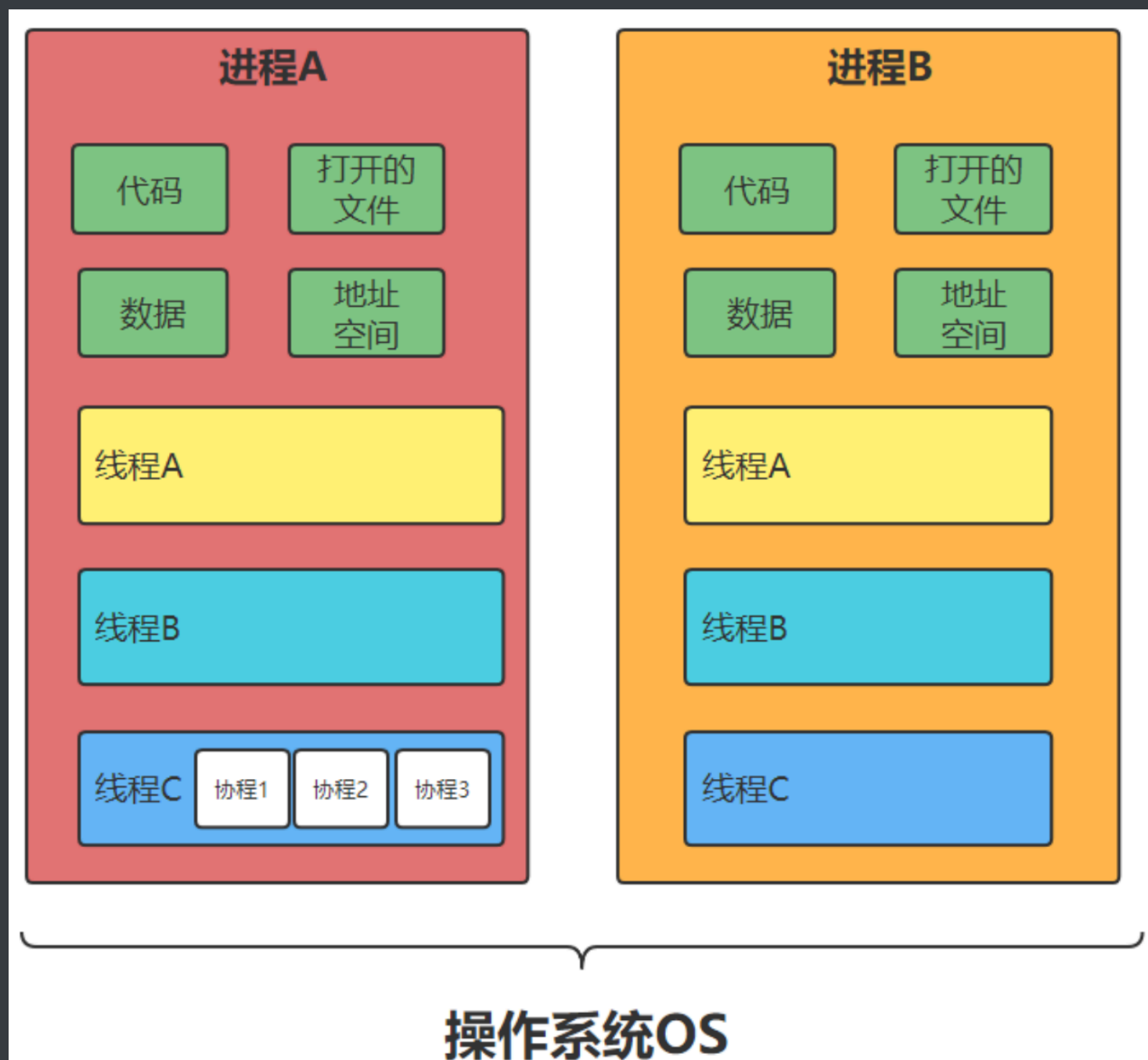
老高



异步书店
老板



线程 Thread



线程Thread特点:

1. 线程（Thread）是操作系统能够进行运算调度的最小单位。它被包含在进程之中，是进程中的实际运作单位
2. 线程是程序执行的最小单位，而进程是操作系统分配资源的最小单位；
3. 一个进程由一个或多个线程组成，线程是一个进程中代码的不同执行路线；
4. 拥有自己独立的栈和共享的堆，共享堆，不共享栈，标准线程由操作系统调度；

5. 调度和切换：线程上下文切换比进程上下文切换要快得多。

线程的创建方式

Python的标准库提供了两个模块：`_thread` 和 `threading`，`_thread` 是低级模块，`threading` 是高级模块，对 `_thread` 进行了封装。绝大多数情况下，我们只需要使用 `threading` 这个高级模块。

线程的创建可以通过两种方式：

1、方法包装

2、类包装

线程的执行统一通过 `start()` 方法启动。

创建线程 – 方法包装

demo01.py ×

```
1 from threading import Thread
2 import threading
3 from time import sleep
4
5
6 def worker(name): 2 用法
7     for i in range(3):
8         print("Thread {0} - {1}".format(*args: name, i))
9         # 可以获取线程名，也就是 Thread 类的 name 参数传递的值
10        # print(threading.current_thread().name)
11        sleep(1)
12
13 if __name__ == '__main__':
14     t1 = Thread(target=worker, name='thread1', args=("thread1",))
15     t2 = Thread(target=worker, name='thread2', args=("thread2",))
16
17     t1.start()
18     t2.start()
19
20     print('done')
```

1 ⌵

运行 demo01 ×

↑
↓
≡
≡
🖨
🗑

D:\proj\threads\.venv\Scr:
Thread thread1 - 0
thread1
Thread thread2 - 0
thread2done

Thread thread1 - 1
thread1
Thread thread2 - 1
thread2
Thread thread1 - 2
thread1
Thread thread2 - 2
thread2

进程已结束，退出代码为 0

Thread **类**

- `target`：指定该线程要执行的可调用对象（这里是函数 `worker`）。

- `name`：给线程起一个名字，方便调试和日志输出。如果不指定，系统会分配默认名称（如 `Thread-1`）。
- `args`：以元组形式传递给 `target` 函数的参数。这里传入 `("thread1",)`，注意元组中只有一个元素时需加逗号。

创建线程 -- 类包装

mo02.py ×

```
from threading import Thread
from time import sleep

class Worker(Thread): 2用法
    def __init__(self, name):
        super().__init__(name = name)
        self.description = '测试类包装线程'

    def run(self):
        for i in range(3):
            # 直接拿到名字 self.name
            print(f'Thread name = {self.name} at {i} - {self.description}')
            sleep(1)

if __name__ == '__main__':
    t1 = Worker(name='demo01')
    t2 = Worker(name='demo02')

    t1.start()
    t2.start()

    print('done')
```

运行 demo02 ×

↑
↓
≡
⌵
📄
🗑

D:\proj\threads\.venv\Scripts\python.exe D:\proj\threads\src\threads\demo02.py
Thread name = demo01 at 0 - 测试类包装线程
Thread name = demo02 at 0 - 测试类包装线程done

Thread name = demo01 at 1 - 测试类包装线程
Thread name = demo02 at 1 - 测试类包装线程
Thread name = demo02 at 2 - 测试类包装线程Thread name = demo01 at 2 - 测试类包装线程

进程已结束，退出代码为 0

join() 和守护线程

join() 函数

在之前的代码中，主线程不会等待子线程结束。（主线程在子线程没有执行完就执行了后续代码，输出了 'done'，之后主线程已经结束了；子线程还在运行。）

如果需要等待子线程结束后再结束主线程，可以使用 `join()` 函数。

```
from threading import Thread
import threading
from time import sleep

def worker(name):
    2 用法
    for i in range(3):
        p = Thread(target=worker, name=name, args=(name, i))
        # i: int 传递的值
        # print(threading.current_thread().name)
        sleep(1)

if __name__ == '__main__':
    t1 = Thread(target=worker, name='thread1', args=("thread1",))
    t2 = Thread(target=worker, name='thread2', args=("thread2",))

    t1.start()
    t2.start()

    ## 在这个案例中增加 join 函数等待子线程结束
    t1.join()
    t2.join()

    print('done')
```

1 ^ v

D:\proj\threads\.venv\Scripts

Thread thread1 - 0
Thread thread2 - 0
Thread thread1 - 1
Thread thread2 - 1
Thread thread1 - 2
Thread thread2 - 2
done

进程已结束，退出代码为 0

守护线程

在行为上还有一种叫守护线程，主要的特征是它的生命周期。主线程死亡，它也就随之死亡。在python中，线程通过 `setDaemon(True|False)` 来设置是否为守护线程，`isDaemon()` 用于获取线程是否为守护线程。

在python3.10之前，上述两个函数可以用来读写守护进程属性；之后的版本已经废弃了这两个函数，可以直接用 `t1.daemon` 来操作属性读写。

守护线程的作用：为其他线程提供便利服务，守护线程最典型的应用就是 GC (垃圾收集器)。

```
from threading import Thread
from time import sleep

class Worker(Thread): 2用法
    def __init__(self, name):
        super().__init__(name = name)
        self.description = '测试类包装线程'

    def run(self):
        for i in range(3):
            # 直接拿到名字 self.name
            print(f'Thread name = {self.name} at {i} - {self.description}')
            sleep(1)

if __name__ == '__main__':
    t1 = Worker(name='demo01')
    t2 = Worker(name='demo02')

    """
    Thread 类的 setDaemon(True) 在 python3.10 之前才有, 高版本已经废弃
    isDaemon() 同理 3.10 之前才有
    高版本直接用 daemon 属性即可读写
    """

    t1.daemon = True
    t2.daemon = True

    print(t1.daemon, t2.daemon)

    t1.start()
    t2.start()

    print('done')
```

D:\proj\threads\.venv\Scripts\python.exe
True True
Thread name = demo01 at 0 - 测试类包装线程
Thread name = demo02 at 0 - 测试类包装线程
done
进程已结束，退出代码为 0

子线程在主线接收后不会再执行了

Python 进程（解释器）退出的唯一硬性条件是：所有非守护线程（non-daemon threads）都已执行完毕。

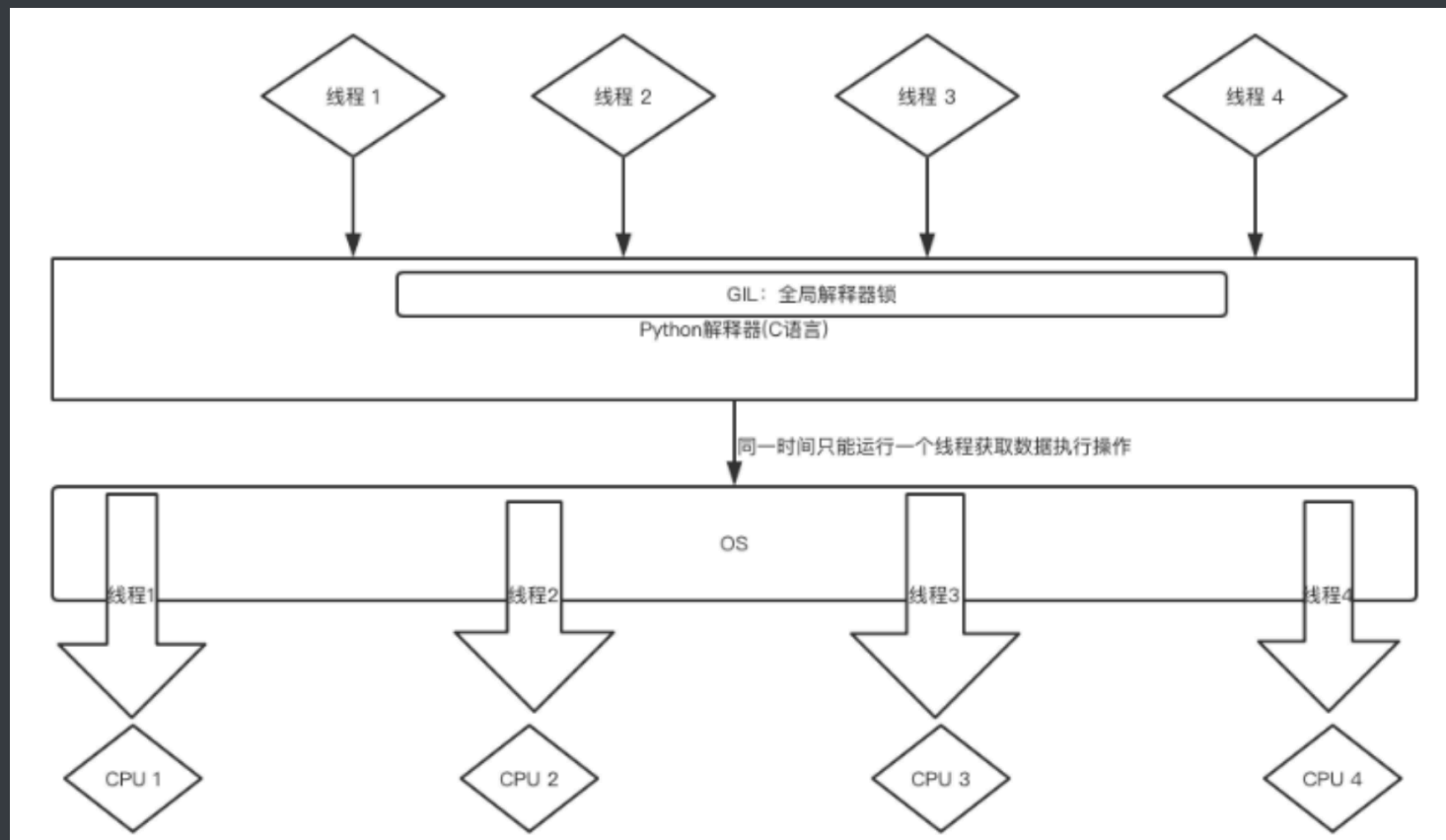
所以，上述两个 t1 t2 线程，只要有一个线程不是守护线程就不会停止。

全局解释器锁 GIL 问题

在python中，无论计算机有多少核，在Cpython解释其中永远都是假象。无论是4核心、8核心等，在同一个时间执行的线程只有一个线程。这是python的一个设计缺陷。

Python GIL(Global Interpreter Lock)

Python代码的执行由Python 虚拟机(也叫解释器主循环，CPython 版本)来控制，Python 在设计之初就考虑到要在解释器的主循环中，同时只有一个线程在执行，即在任意时刻，只有一个线程在解释器中运行。对Python 虚拟机的访问由全局解释器锁（GIL）来控制，正是这个锁能保证同一时刻只有一个线程在运行。



线程同步和互斥锁

线程同步概念

处理多线程问题时，多个线程访问同一个对象，并且某些线程还想着修改这个对象。这时候就需要用到“线程同步”。

线程同步其实就是一种等待机制，多个线程需要同时访问此对象的线程进入这个对象的等待池形成队列，等待前面的线程使用完毕后，下一个线程再使用。

- 以下是一个没有同步的案例，对同一个账户 acc 进行操作，就可能会出问题。

```
from threading import Thread
from time import sleep

class Account: 1 个用法
    def __init__(self, money, name):
        self.money = money
        self.name = name

#模拟提款操作
class Drawing(Thread): 2 用法
    def __init__(self, drawingNum, account):
        Thread.__init__(self)
        self.drawingNum = drawingNum
        self.account = account
        self.expenseTotal = 0

    def run(self):
        if self.account.money - self.drawingNum < 0:
            return

        sleep(1) # 判断完后阻塞。其他线程开始运行。
        self.account.money -= self.drawingNum
        self.expenseTotal += self.drawingNum

        print(f"账户: {self.account.name}, 余额是: {self.account.money}")
        print(f"账户: {self.account.name}, 总共取了: {self.expenseTotal}")

if __name__ == '__main__':
    a1 = Account( money: 100, name: "jerry")
    draw1 = Drawing( drawingNum: 80, a1) # 定义取钱线程对象;
    draw2 = Drawing( drawingNum: 80, a1) # 定义取钱线程对象;
    draw1.start() # 你取钱
    draw2.start() # 另一个人取钱
```

D:\proj\threads\.venv\Scr
账户: jerry, 余额是: 20
账户: jerry, 总共取了: 80
账户: jerry, 余额是: -60
账户: jerry, 总共取了: 80
进程已结束, 退出代码为 0

互斥锁机制

可以通过“锁机制”来实现线程同步问题，锁机制有如下几个要点：

- 必须使用同一个锁对象
- 互斥锁的作用就是保证同一时刻只能有一个线程去操作共享数据，保证共享数据不会出现错误问题
- 使用互斥锁的好处确保某段关键代码只能由一个线程从头到尾完整地去执行
- 使用互斥锁会影响代码的执行效率
- 同时持有多把锁，容易出现死锁的情况

互斥锁是什么？

互斥锁: 对共享数据进行锁定，保证同一时刻只能有一个线程去操作。

注意: 互斥锁是多个线程一起去抢, 抢到锁的线程先执行, 没有抢到锁的线程需要等待, 等互斥锁使用完释放后, 其它等待的线程再去抢这个锁。

threading 模块中定义了 Lock 变量, 这个变量本质上是一个函数, 通过调用这个函数可以获取一把互斥锁。

```
from threading import Thread, Lock
from time import sleep

class Account: 1 个用法
    def __init__(self, money, name):
        self.money = money
        self.name = name

class Drawing(Thread): 2 用法
    def __init__(self, drawingNum, account):
        Thread.__init__(self)
        self.drawingNum = drawingNum
        self.account = account
        self.expenseTotal = 0

    def run(self):
        lock.acquire()

        if self.account.money - self.drawingNum < 0:
            lock.release()
            return

        sleep(1) # 判断完后阻塞。其他线程开始运行。
        self.account.money -= self.drawingNum
        self.expenseTotal += self.drawingNum
        print(f"账户: {self.account.name}, 余额是: {self.account.money}")
        print(f"账户: {self.account.name}, 总共取了: {self.expenseTotal}")
        lock.release()

if __name__ == '__main__':
    a1 = Account( money: 100, name: "jerry")
    draw1 = Drawing( drawingNum: 80, a1) # 定义取钱线程对象
    draw2 = Drawing( drawingNum: 80, a1) # 定义取钱线程对象
    lock = Lock()
    draw1.start() # 你取钱
    draw2.start() # 另一个人取钱
```

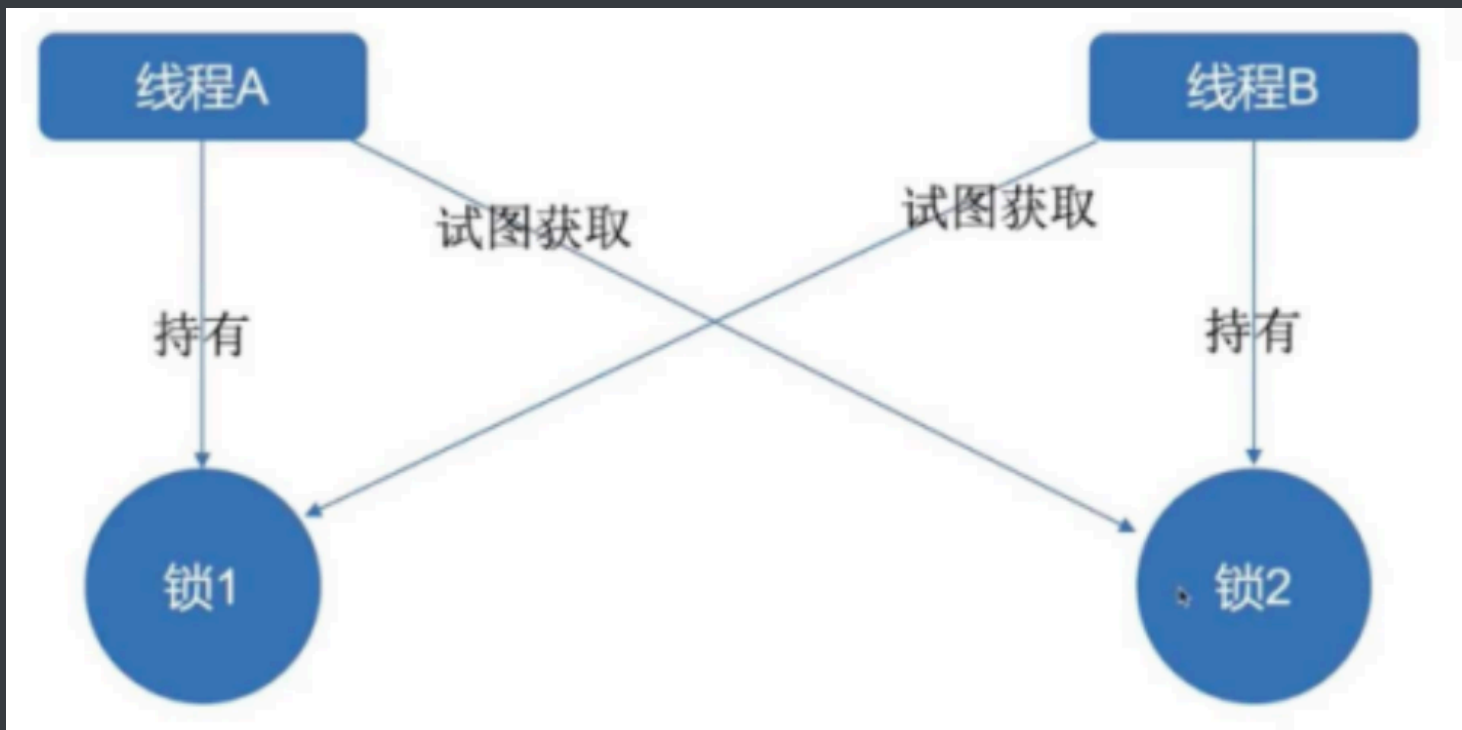
锁的范围要把操作共享数据的地方全包住

D:\proj\threads\.venv\S
账户: jerry, 余额是: 20
账户: jerry, 总共取了: 80
进程已结束, 退出代码为 0

死锁

在多线程程序中, 死锁问题很大一部分是由于一个线程同时获取多个锁造成的。

同一个代码块, 不要同时持有两个对象锁。



```
from threading import Thread, Lock
from time import sleep
def fun1(): 1个用法
    lock1.acquire()
    print('fun1拿到菜刀')
    sleep(2)
    lock2.acquire()
    print('fun1拿到锅')
    lock2.release()

    print('fun1释放锅')
    lock1.release()
    print('fun1释放菜刀')

def fun2(): 1个用法
    lock2.acquire()
    print('fun2拿到锅')
    lock1.acquire()
    print('fun2拿到菜刀')
    lock1.release()
    print('fun2释放菜刀')
    lock2.release()
    print('fun2释放锅')

if __name__ == '__main__':
    lock1 = Lock()
    lock2 = Lock()
    t1 = Thread(target=fun1)
    t2 = Thread(target=fun2)
    t1.start()
    t2.start()
```

死锁，程序卡住

D:\proj\threads\.venv\Scripts\pyt
fun1拿到菜刀
fun2拿到锅

信号量 Semaphore

互斥锁使用后，一个资源同时只有一个线程访问。如果某个资源，我们同时想让N个(指定数值)线程访问？这时候，可以使用信号量。

信号量控制同时访问资源的数量。信号量和锁相似，锁同一时间只 允许一个对象(进程)通过，信号量同一时间允许多个对象(进程)通过。

应用场景：

1. 在读写文件的时候，一般只能只有一个线程在写，而读可以有多个线程同时进行，如果需要限制同时读文件的线程个数，这时候就可以用到信号量了（如果用互斥锁，就是限制同一时刻只能有一个线程读取文件）。
2. 在做爬虫抓取数据时。

底层原理：

信号量底层就是一个内置的计数器。每当资源获取时(调用`acquire`) 计数器-1，资源释放时(调用`release`) 计数器 +1。

```
from threading import Thread, Lock
from time import sleep
from multiprocessing import Semaphore

"""
    一个房间一次只允许两个人通过
    若不使用信号量，会造成所有人都进入这个房子
    若只允许一人通过可以用锁-Lock()
"""

def home(name, se): 1 个用法
    se.acquire()

    print(f'{name} 进入房间')
    sleep(3)
    print(f'{name} 走出房间')

    se.release()

if __name__ == '__main__':
    se = Semaphore(2)
    for i in range(7):
        t = Thread(target=home, args=(f'tom{i}', se))
        t.start()
```

D:\proj\threads\.venv\Scripts
tom0 进入房间
tom1 进入房间
tom1 走出房间tom0 走出房间
tom2 进入房间tom3 进入房间
tom3 走出房间tom2 走出房间
tom4 进入房间tom5 进入房间
tom4 走出房间
tom5 走出房间
tom6 进入房间
tom6 走出房间
进程已结束，退出代码为 0

事件 Event

事件 Event 主要用于唤醒正在阻塞等待状态的线程。

原理

- Event 对象包含一个可由线程设置的信号标志，它允许线程等待某些事件的发生。
- 在初始情况下，event 对象中的信号标志被设置假。如果有线程等待一个 event 对象，而这个 event 对象的标志为假，那么这个线程将会被一直阻塞直至该标志为真。
- 一个线程如果将一个 event 对象的信号标志设置为真，它将唤醒所有等待个 event 对象的线程。如果一个线程等待一个已经被设置为真的 event 对象，那么它将忽略这个事件，继续执行。

Event() 可以创建一个事件管理标志，该标志 (event) 默认为 False，event 对象主要有四种方法可以调用：

方法名	说明
event.wait(timeout=None)	调用该方法的线程会被阻塞，如果设置了timeout参数，超时后，线程会停止阻塞继续执行；
event.set()	将event的标志设置为True，调用wait方法的所有线程将被唤醒
event.clear()	将event的标志设置为False，调用wait方法的所有线程将被阻塞
event.is_set()	判断event的标志是否为True

```
import threading
import time

def chihuoguo(name): 2 用法
    print(f'{name}已经启动')
    print(f'小伙伴{name}已经进入就餐状态')

    event.wait() # 等待通知

    print(f'{name}收到通知了.')
    print(f'小伙伴{name}开始吃咯! ')

if __name__ == '__main__':
    event = threading.Event()

    t1 = threading.Thread(target=chihuoguo, args=('tom',))
    t2 = threading.Thread(target=chihuoguo, args=('cherry',))

    t1.start()
    t2.start()

    time.sleep(5)

    print('--->>5秒后，主线程通知小伙伴开吃')
    event.set() # 发送通知
```

D:\proj\threads\.venv\Scripts\python.exe

tom已经启动
小伙伴tom已经进入就餐状态
cherry已经启动
小伙伴cherry已经进入就餐状态
--->>5秒后，主线程通知小伙伴开吃
tom收到通知了。
cherry收到通知了。
小伙伴tom开始吃咯！小伙伴cherry开始吃咯！

进程已结束，退出代码为 0

生产者和消费者模式

多线程环境下，经常需要多个线程的并发和协作。这个时候，就需要了解一个重要的多线程并发协作模型“生产者/消费者模式”。



1. 生产者

生产者指的是负责生产数据的模块（这里模块可能是：方法、对象、线程、进程）。

2. 消费者

消费者指的是负责处理数据的模块（这里模块可能是：方法、对象、线程、进程）。

3. 缓冲区

消费者不能直接使用生产者的数据，它们之间有个“缓冲区”。生产者将生产好的数据放入“缓冲区”，消费者从“缓冲区”拿要处理的数据。

缓冲区是实现并发的核心，缓冲区的设置有3个好处：

1. 实现线程的并发协作

实现线程的并发协作有了缓冲区以后，生产者线程只需要往缓冲区里面放置数据，而不需要管消费者消费的情况；同样，消费者只需要从缓冲区拿数据处理即可，也不需要管生产者生产的情况。这样，就从逻辑上实现了“生产者线程”和“消费者线程”的分离。

2. 解耦了生产者和消费者

生产者不需要和消费者直接打交道。

3. 解决忙闲不均，提高效率

生产者生产数据慢时，缓冲区仍有数据，不影响消费者消费；消费者处理数据慢时，生产者仍然可以继续往缓冲区里面放置数据。

缓冲区和queue对象：

从一个线程向另一个线程发送数据最安全的方式可能就是用 `queue` 库中的队列了。

创建一个被多个线程共享的 `queue` 对象，这些线程通过使用 `put()` 和 `get()` 操作来向队列中添加或者删除元素。

`queue` 对象已经包含了必要的锁，所以可以通过它在多个线程间安全地共享数据。

```
from queue import Queue
from threading import Thread
from time import sleep

def producer(): 1 个用法
    id = 1
    while True:
        if queue.qsize() < queue.maxsize:
            print(f'生产馒头 {id} 号')
            queue.put(f'大馒头-{id}号')
            id += 1
        else:
            print('馒头框满了, 需要等待消费')

        sleep(1)
```

```
def consumer(): 2 用法
    while True:
        print(f'吃馒头 -> {queue.get()}')
        sleep(1)
```

```
if __name__ == '__main__':
    queue = Queue(maxsize=3)
    producer = Thread(target=producer)
    consumer1 = Thread(target=consumer)
    consumer2 = Thread(target=consumer)

    producer.start()
    sleep(5)
    consumer1.start()
    consumer2.start()
```



D:\proj\threads\.venv\Scr

生产馒头 1 号

生产馒头 2 号

生产馒头 3 号

馒头框满了, 需要等待消费

馒头框满了, 需要等待消费

吃馒头 -> 大馒头-1号

吃馒头 -> 大馒头-2号

生产馒头 4 号

吃馒头 -> 大馒头-3号

吃馒头 -> 大馒头-4号

生产馒头 5 号

吃馒头 -> 大馒头-5号

生产馒头 6 号

吃馒头 -> 大馒头-6号

生产馒头 7 号

吃馒头 -> 大馒头-7号

生产馒头 8 号

吃馒头 -> 大馒头-8号

生产馒头 9 号

吃馒头 -> 大馒头-9号

生产馒头 10 号

吃馒头 -> 大馒头-10号

生产馒头 11 号

吃馒头 -> 大馒头-11号

生产馒头 12 号

吃馒头 -> 大馒头-12号

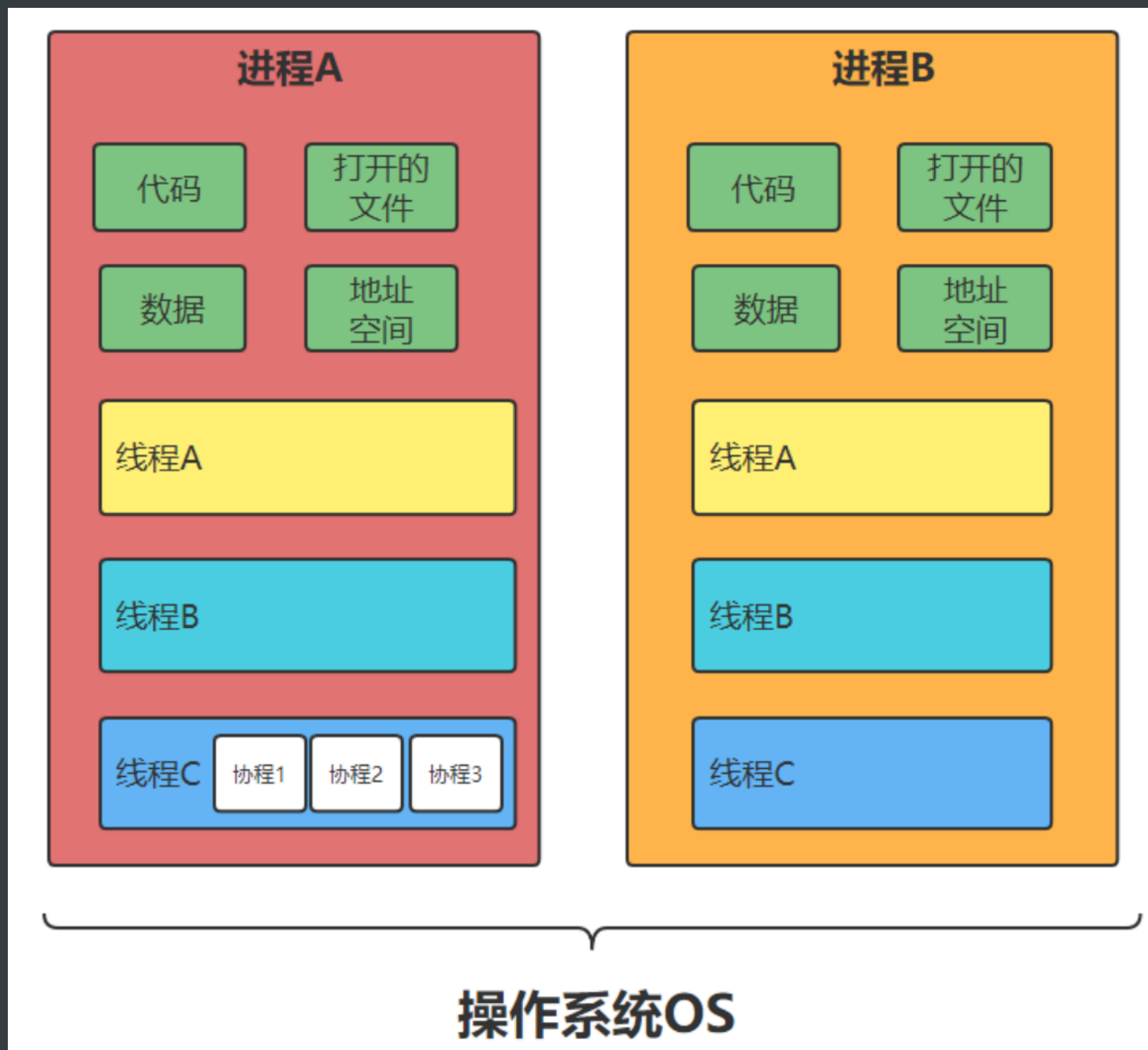
生产馒头 13 号

吃馒头 -> 大馒头-13号

生产馒头 14 号

吃馒头 -> 大馒头-14号

进程 Process



进程Process: 拥有自己独立的栈和堆，既不共享堆，也不共享栈。进程由操作系统调度；进程切换需要的资源很大，效率低。

进程优点:

1. 可以使用计算机多核，进行任务的并行执行，提高执行效率
2. 运行不受其它进程影响，创建方便
3. 空间独立，数据安全

进程缺点：

进程的创建和删除消耗的系统资源较多。

进程的创建方式

Python的标准库提供了个模块： `multiprocessing`

创建进程后，需要使用 `start()` 启动进程

创建进程 -- 方法模式

```
from multiprocessing import Process
from time import sleep
import os

def work(name):
    print('当前进程ID =', os.getpid())
    print('父进程ID =', os.getppid())
    print(f'process name={name} start')
    sleep(2)
    print(f'process name={name} end')

if __name__ == '__main__':
    print('当前主进程ID =', os.getpid())
    p1 = Process(target=work, args=('p1',))
    p2 = Process(target=work, args=('p2',))

    p1.start()
    p2.start()
```

获取当前进程 pid
获取父进程 pid

```
D:\proj\threads\.venv\Scripts>python main.py
当前主进程ID = 10564
当前进程ID = 26636
父进程ID = 10564
process name=p1 start
当前进程ID = 21268
父进程ID = 10564
process name=p2 start
process name=p1 end
process name=p2 end

进程已结束，退出代码为 0
```

创建进程 -- 进程 `Process` 类

和使用 `Thread` 类创建子线程的方式非常类似，使用 `Process` 类创建实例化对象，其本质是调用该类的构造方法创建新进程。

`Process` 类的构造方法格式如下：

```
def __init__(self, group=None, target=None, name=None, args=(), kwargs={})
```

- `group` : 该参数未进程实现, 不需要传参;
- `target` : 为新建进程指定执行任务, 也就是指定一个函数;
- `name` : 为新建进程设置名称;
- `args` : 为 `target` 参数指定的参数传递非关键字参数;
- `kwargs` : 为 `target` 参数指定的参数传递关键字参数。

```
from multiprocessing import Process
from time import sleep

class MyProcess(Process): 2 用法
    def __init__(self, name):
        super().__init__()
        self.name = name

    def run(self):
        print(f'process name={self.name} start')
        sleep(2)
        print(f'process name={self.name} end')

if __name__ == '__main__':
    p1 = MyProcess('p1')
    p2 = MyProcess('p2')

    p1.start()
    p2.start()
```

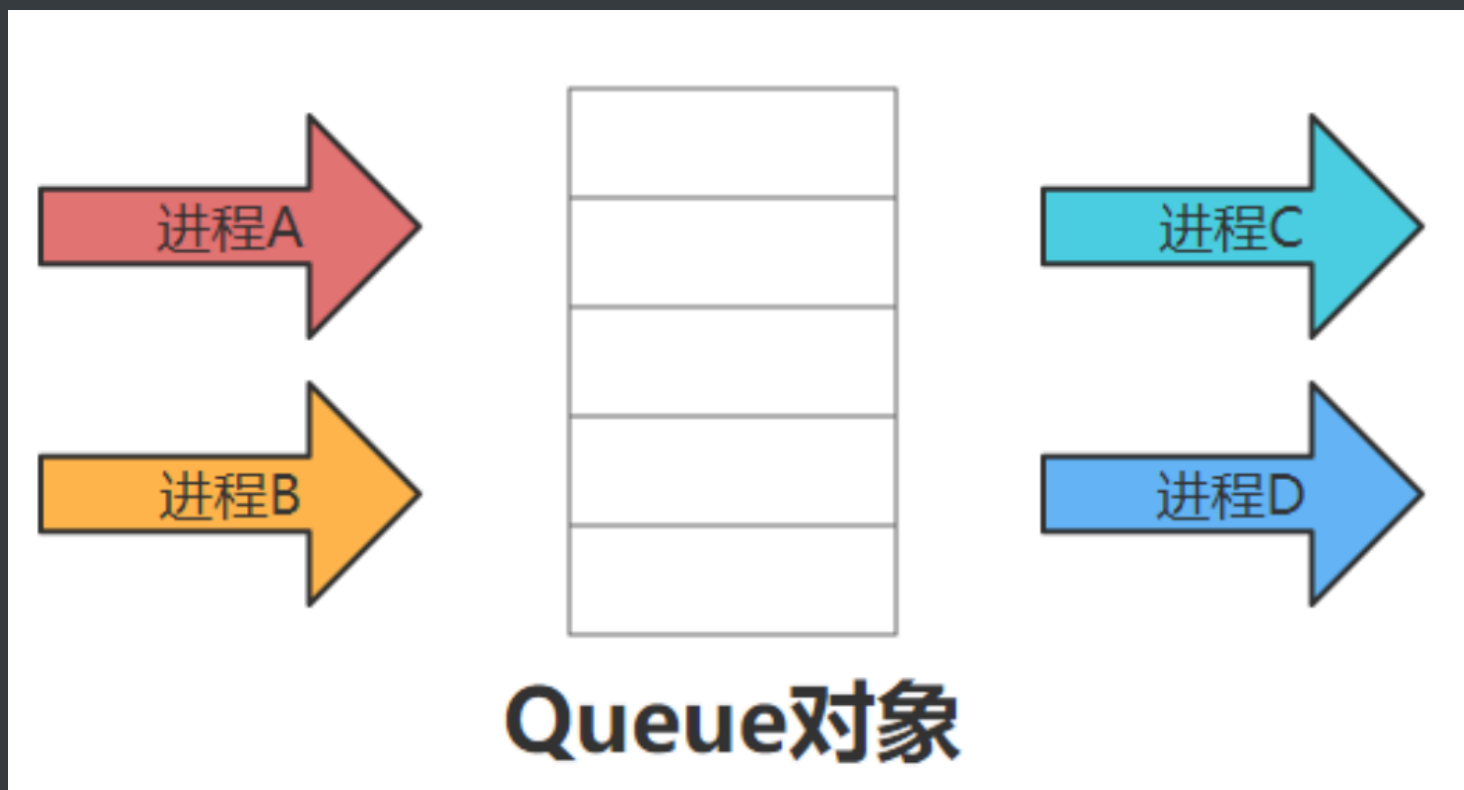


```
D:\proj\threads\.venv\S
process name=p1 start
process name=p2 start
process name=p1 end
process name=p2 end
```

```
进程已结束, 退出代码为 0
|
```

进程通信

Queue 实现进程间通信



在线程的生产者和消费者模式中使用了 `queue` 模块中的 `Queue` 队列类。

进程间通信需要使用 `multiprocessing` 模块中的 `Queue` 类。

简单理解 `Queue` 实现进程间通信的方式，就是使用了操作系统给开辟的一个队列空间，各个进程可以把数据放到该队列中，当然也可以从队列中把自己需要的信息取走。

- 使用 `Queue` 实现进程间通信的经典代码

```

from multiprocessing import Process, Queue

class CommunicateProcess(Process):
    def __init__(self, name, mq):
        super().__init__()
        self.name = name
        self.mq = mq

    def run(self):
        print(f'process name={self.name} start')
        # 读取队列中的数据，并从队列中取出
        print('---- {0} ----'.format(self.mq.get()))
        # 往队列中写入数据
        self.mq.put(self.name)
        print(f'process name={self.name} end')

if __name__ == '__main__':
    mq = Queue()
    mq.put(1)
    mq.put(2)
    mq.put(3)

    p_list = []
    for i in range(3):
        p = CommunicateProcess('p{}'.format(i), mq)
        p.start()
        p_list.append(p)

    # 等待进程结束
    for p in p_list:
        p.join()

    print(mq.get())
    print(mq.get())
    print(mq.get())

```



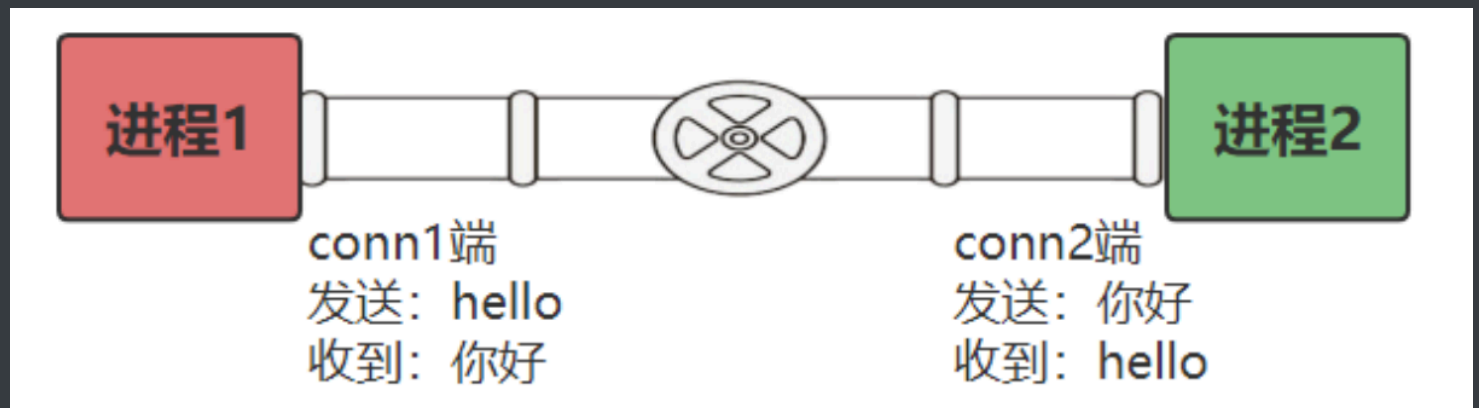
```

D:\proj\threads\.venv\Scr
process name=p0 start
---- 1 ----
process name=p0 end
process name=p1 start
---- 2 ----
process name=p1 end
process name=p2 start
---- 3 ----
process name=p2 end
p0
p1
p2

```

进程已结束，退出代码为 0

Pipe 实现进程间通信



```
import multiprocessing
from time import sleep

def func1(conn1): 1 个用法
    sub_info = "Hello!"
    print(f"进程1--{multiprocessing.current_process().pid}发送数据: {sub_info}")
    sleep(1)
    conn1.send(sub_info)
    print(f"来自进程2:{conn1.recv()}")
    sleep(1)

def func2(conn2): 1 个用法
    sub_info = "你好!"
    print(f"进程2--{multiprocessing.current_process().pid}发送数据: {sub_info}")
    sleep(1)
    conn2.send(sub_info)
    print(f"来自进程1:{conn2.recv()}")
    sleep(1)

if __name__ == '__main__':
    #创建管道, 返回管道的两端
    conn1, conn2 = multiprocessing.Pipe()
    # 创建2个子进程, 各自拿管道的一端用于通信
    process1 = multiprocessing.Process(target=func1, args=(conn1,))
    process2 = multiprocessing.Process(target=func2, args= (conn2,))
    # 启动子进程
    process1.start()
    process2.start()
```

D:\proj\threads\.venv\Script
进程1--18484发送数据: Hello!
进程2--29652发送数据: 你好!
来自进程2:你好!
来自进程1:Hello!
进程已结束, 退出代码为 0

Manager 管理器

管理器提供了一种创建共享数据的方法, 从而可以在不同进程中共享。

它是 Python 多进程模块中用于在进程间共享复杂数据的高级工具。它的核心作用是创建一个“代理服务器”进程, 让多个子进程通过这个代理来安全、方便地操作同一份数据。

Manager (管理器)

支持几乎所有类型：列表、字典、集合、自定义类实例、嵌套结构

较慢（需要序列化+网络/管道传输）

需要灵活数据结构、少量数据共享、跨网络分布式

```
from multiprocessing import Process, current_process
from multiprocessing import Manager

def func(name, m_list, m_dict): 1 个用法
    m_dict[name] = '子进程 execute'
    m_list.append('你好')

if __name__ == '__main__':
    with Manager() as manager:
        m_list = manager.list()
        m_dict = manager.dict()

        m_list.append('main process')

        p = Process(target=func, args=('p1', m_list, m_dict))
        p.start()
        p.join()

        print(m_list)
        print(m_dict)
```

```
D:\proj\threads\.venv\Scr
['main process', '你好']
{'p1': '子进程 execute'}

进程已结束，退出代码为 0
```

进程池 Process Pool

进程池可以提供指定数量的进程给用户使用，即当有新的请求提交到进程池中时，如果池未满，则会创建一个新的进程用来执行该请求；反之，如果池中的进程数已经达到规定最大值，那么该请求就会等待，只要池中有进程空闲下来，该请求就能得到执行。

使用进程池的优点：

- 1. 提高效率，节省开辟进程和开辟内存空间的时间以及销毁进程的时间
- 2. 节省内存空间

类/方法	功能	参数
<code>Pool(processes)</code>	创建进程池对象	<code>processes</code> 表示进程池中有多少进程
<code>pool.apply_async(func,args,kwds)</code>	异步执行 ； 将事件放入到进程池队列	<code>func</code> 事件函数 <code>args</code> 以元组形式给 <code>func</code> 传参 <code>kwds</code> 以字典形式给 <code>func</code> 传参 返回值：返回一个代表进程池事件的对象，通过返回值的 <code>get</code> 方法可以得到事件函数的返回值
<code>pool.apply(func,args,kwds)</code>	同步执行； 将事件放入到进程池队列	<code>func</code> 事件函数 <code>args</code> 以元组形式给 <code>func</code> 传参 <code>kwds</code> 以字典形式给 <code>func</code> 传参
<code>pool.close()</code>	关闭进程池	
<code>pool.join()</code>	回收进程池	
<code>pool.map(func,iter)</code>	类似于python的map函数， 将要做的 事件放入进程池	<code>func</code> 要执行的函数 <code>iter</code> 迭代对象

1. 进程池使用案例

```
from multiprocessing import Pool
import os
from time import sleep

def func1(name):
    print(f'current process id is {os.getpid()}, process name is {name}')
    sleep(2)
```

```

    return name

def func2(args):
    print(args)

if __name__ == '__main__':
    # 1、创建进程池，processes 可以指定这个池中最大能有多少个进程
    pool = Pool(processes=4)

    # 2、===== 带有回调函数的异步任务 =====
    # apply_async: 异步执行函数，不阻塞主进程
    # func: 要执行的函数
    # args: 传递给func的参数，必须是元组形式（注意逗号）
    # callback: 回调函数，当func执行完成后自动调用，并传入func的返回值

    ...

    执行流程:
    pool.apply_async(func=func1, args=('sxt1',), callback=func2)
    a. 从进程池中分配一个子进程执行 func1('sxt1')
    b. 主进程继续往下执行，不等待 func1 完成
    c. func1 执行完成后，自动调用 func2，并传入 func1 的返回值作为参数
    d. func2打印 'sxt1'
    ...

    pool.apply_async(func=func1, args=('sxt1',), callback=func2)
    pool.apply_async(func=func1, args=('sxt2',), callback=func2)
    pool.apply_async(func=func1, args=('sxt3',), callback=func2)

    ...

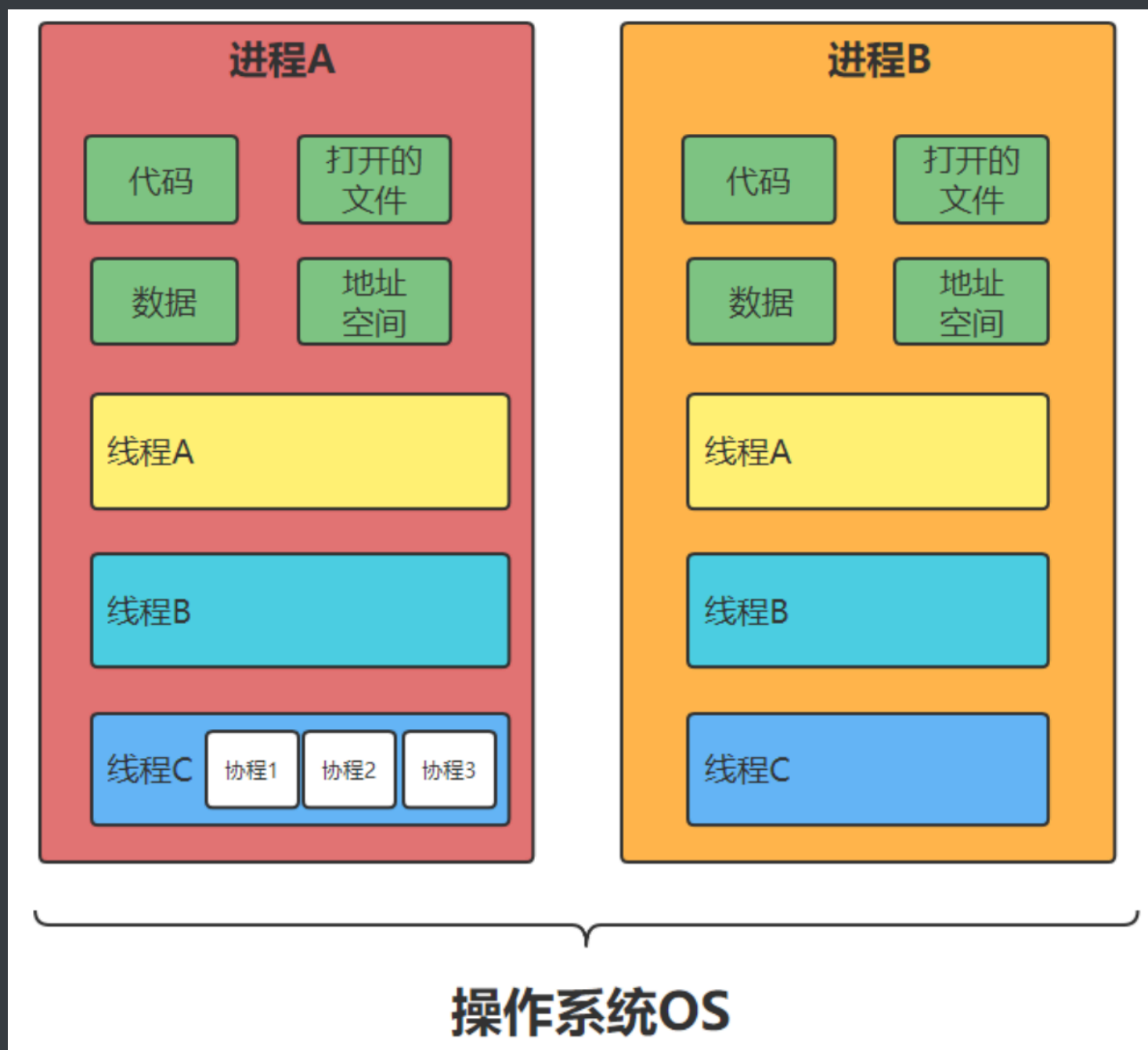
    ===== 不带回调函数的异步任务 =====
    没有callback参数，意味着任务执行完成后不会自动处理返回值
    但任务仍然会正常执行
    ...

    pool.apply_async(func=func1, args=('sxt4',))
    pool.apply_async(func=func1, args=('sxt5',))
    pool.apply_async(func=func1, args=('sxt6',))

    # close(): 关闭进程池，不再接受新的任务

```


协程 Coroutines



协程 Coroutines，也叫做纤程（Fiber）

协程，全称是“协同程序”，用来实现任务协作。

是一种在线程中，比线程更加轻量级的存在，由程序员自己写程序来管理。

当出现IO阻塞时，CPU一直等待IO返回，处于空转状态。这时候用协程，可以执行其他任务。当IO返回结果后，再回来处理数据。充分利用了IO等待的时间，提高了效率。

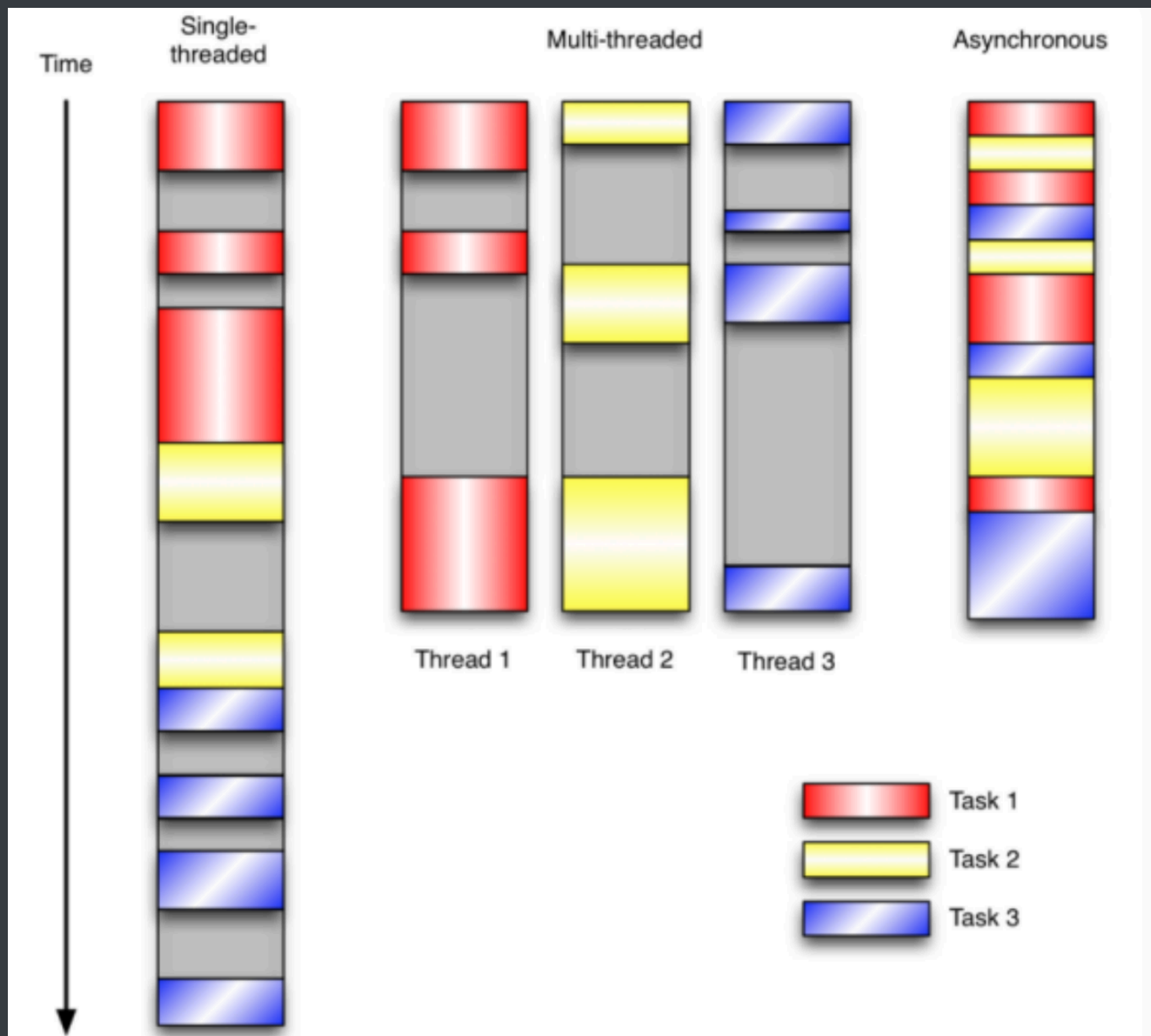
协程

1. 协程的核心（控制流的让出和恢复，即保留和恢复现场）

- 每个协程有自己的执行栈，可以保存自己的执行现场
- 可以由用户程序按需创建协程（比如：遇到io操作）
- 协程“主动让出（yield）”执行权时候，会保存执行现场(保存中断时的寄存器上下文和栈)，然后切换 到其他协程
- 协程恢复执行（resume）时，根据之前保存的执行现场恢复到中断前的状态，继续执行，这样就 通过协程实现了轻量的由用户态调度的多任务模型

2. 协程和多线程比较

比如，有3个任务需要完成，每个任务都在等待I/O操作时阻塞自身。阻塞在I/O操作上所花费的时间已经用灰色框标示出来了。



- 在单线程同步模型中，任务按照顺序执行。如果某个任务因为I/O而阻塞，其他所有的任务都必须等待，直到它完成之后它们才能依次执行。
- 多线程版本中，这3个任务分别在独立的线程中执行。这些线程由操作系统来管理，在多处理器系统上可以并行处理，或者在单处理器系统上交错执行。这使得当某个线程阻塞在某个资源的同时其他线程得以继续执行。
- 协程版本的程序中，3个任务交错执行，但仍然在一个单独的线程控制中。当处理I/O或者其他昂贵的操作时，注册一个回调到事件循环中，然后当I/O操作完成时继续执行。回调描述了该如何处理某个事件。事件循环轮询所有的事件，当事件到来时将它们分配给等待处理事件的回调函数。

3. 协程的优点

- 由于自身带有上下文和栈，无需线程上下文切换的开销，属于程序级别的切换，操作系统完全感知不到，因而更加轻量级；
- 无需原子操作的锁定及同步的开销；
- 方便切换控制流，简化编程模型；
- 单线程内就可以实现并发的效果，最大限度地利用cpu，且可扩展性高，成本低（注：一个CPU支持上万的协程都不是问题。所以很适合用于高并发处理）

4. 协程的缺点

- 无法利用多核资源：协程的本质是个单线程，它不能同时将单个CPU 的多个核用上，协程需要和进程配合才能运行在多CPU上。
- 当然我们日常所编写的绝大部分应用都没有这个必要，除非是cpu密集型应用。

使用 yield 实现协程（已淘汰）

Python中的协程经历了很长的一段发展历程。其大概经历了如下三个阶段：

- 最初的生成器变形 `yield/send`
- 引入 `@asyncio.coroutine` 和 `yield from`
- Python3.5版本后，引入 `async/await` 关键字

基于yield并发执行，多任务之间来回切换，这就是个简单的协程的体现，但是他能够节省I/O时间吗？不能。

```
import time

def func1(): 1 个用法
    for i in range(3):
        print(f'北京: 第{i}次打印啦')
        time.sleep(1)

    return "func1执行完毕"

def func2(): 1 个用法
    for k in range(3):
        print(f'上海: 第{k}次打印了' )
        time.sleep(1)

    return "func2执行完毕"

def main(): 1 个用法
    func1()
    func2()

if __name__ == '__main__':
    start_time = time.time()
    main()
    end_time = time.time()
    print(f"耗时{end_time-start_time}") # 不使用协程, 耗时6秒
```

单线程串行任务



D:\proj\threads\.venv

北京: 第0次打印啦

北京: 第1次打印啦

北京: 第2次打印啦

上海: 第0次打印了

上海: 第1次打印了

上海: 第2次打印了

耗时6.001826286315918

进程已结束, 退出代码为 0

```
import time

def func1(): 1个用法
    for i in range(3):
        print(f'北京: 第{i}次打印啦')
        yield # 只要方法包含了 yield 就变成了一个生成器
        time.sleep(1)

    return "func1执行完毕"

def func2(): 1个用法
    g = func1() # func1 是一个生成器, func1() 就不会直接调用, 需要通过 next() 或 for 循环调用
    for k in range(3):
        print(f'上海: 第{k}次打印了' )
        next(g)
        time.sleep(1)

    return "func2执行完毕"

if __name__ == '__main__':
    start_time = time.time()
    func2()
    end_time = time.time()
    print(f"耗时{end_time-start_time}")
```

D:\proj\threads\.venv\
上海: 第0次打印了
北京: 第0次打印啦
上海: 第1次打印了
北京: 第1次打印啦
上海: 第2次打印了
北京: 第2次打印啦
耗时5.001458168029785

进程已结束, 退出代码为 0

Asyncio 实现协程（重点）

1. 正常的函数执行时是不会中断的, 所以要写一个能够中断的函数, 就需要加 `async`
2. `async` 用来声明一个函数为异步函数, 异步函数的特点是能在函数执行过程中挂起, 去执行其他异步函数, 等到挂起条件（假设挂起条件是 `sleep(5)`）消失后, 也就是5秒到了再回来执行
3. `await` 用来用来声明程序挂起, 比如异步程序执行到某一步时需要等待的时间很长, 就将此挂起, 去执行其他的异步程序。
4. `asyncio` 是python3.5之后的协程模块, 是python实现并发重要的包, 这个包使用事件循环驱动实现并发。

1. `async def` 定义异步函数
2. `await` 等待异步操作
3. `asyncio.gather()` 并发执行多个任务
4. `asyncio.run()` 运行异步程序, 负责调用异步函数
5. `asyncio.sleep()` 异步等待, 等待时不阻塞其它任务


```
# 3. 所有任务完成后, gather 返回结果列表
res = await asyncio.gather(func1(), func2())
# await 异步执行 func1 方法
# 返回值为函数的返回值列表, 本例为 ['func1执行完毕', 'func2执行完毕']
print(res)

if __name__ == '__main__':
    start_time = time.time()
    asyncio.run(main())
    end = time.time()
    # 只有 3s, 效率非常高,
    # 串行需要 6s 多
    # yield方式的协程也需要 5s 多
    print(f'耗时 {end - start_time} s')
```

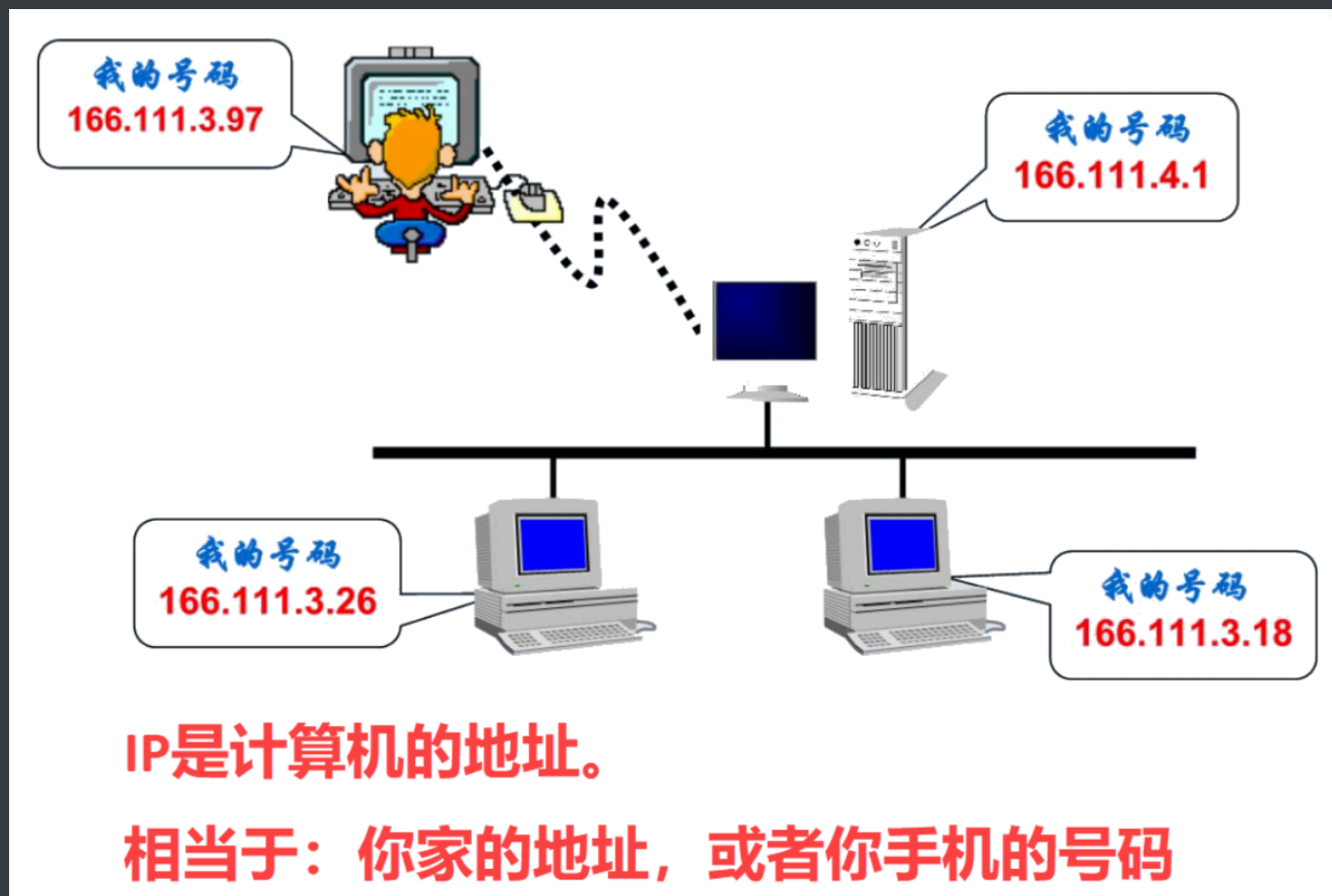
网络编程

网络编程概念

网络编程：解决两台计算机之间如何通信。

Python语言中，提供了大量的内置模块和第三方模块用于支持各种网络访问，而且Python语言在网络通信方面的优点特别突出，远远领先其他语言。

IP 地址



IP是Internet Protocol Address，即“互联网协议地址”。

IP用来标识网络中的一个通信实体的地址。

通信实体可以是计算机、路由器等。比如互联网的每个服务器都要有自己的IP地址，而每个局域网的计算机要通信也要配置IP地址。

路由器是连接两个或多个网络的网络设备。

类别	最大网络数	IP地址范围	单个网段最大主机数	私有IP地址范围
A	126(2^7-2)	1.0.0.1-127.255.255.254	16777214	10.0.0.0-10.255.255.255
B	16384(2^14)	128.0.0.1-191.255.255.254	65534	172.16.0.0-172.31.255.255
C	2097152(2^21)	192.0.0.1-223.255.255.254	254	192.168.0.0-192.168.255.255

目前主流使用的IP地址是IPV4，但是随着网络规模的不断扩大，IPV4面临着枯竭的危险，所以推出了IPV6。

IP地址实际上是一个32位整数（称为IPv4），以字符串表示的IP地址如 192.168.0.1 实际上是把32位整数按8位分组后的数字表示，目的是便于阅读。

IPv6地址实际上是一个128位整数，它是目前使用的IPv4的升级版，以字符串表示类似于 2001:0db8:85a3:0042:1000:8a2e:0370:7334

公有地址

公有地址 (Public address) 由Inter NIC (Internet Network Information Center互联网信息中心) 负责。这些IP地址分配给注册并向Inter NIC提出申请的组织机构。通过它直接访问互联网。

私有地址

私有地址 (Private address) 属于非注册地址，专门为组织机构内部使用。

以下列出留用的内部私有地址

A类 10.0.0.0--10.255.255.255

B类 172.16.0.0--172.31.255.255

C类 192.168.0.0--192.168.255.255

① windows下，我们可以通过命令 `ipconfig` 获取网卡信息。（Linux和Mac，是 `ifconfig`）

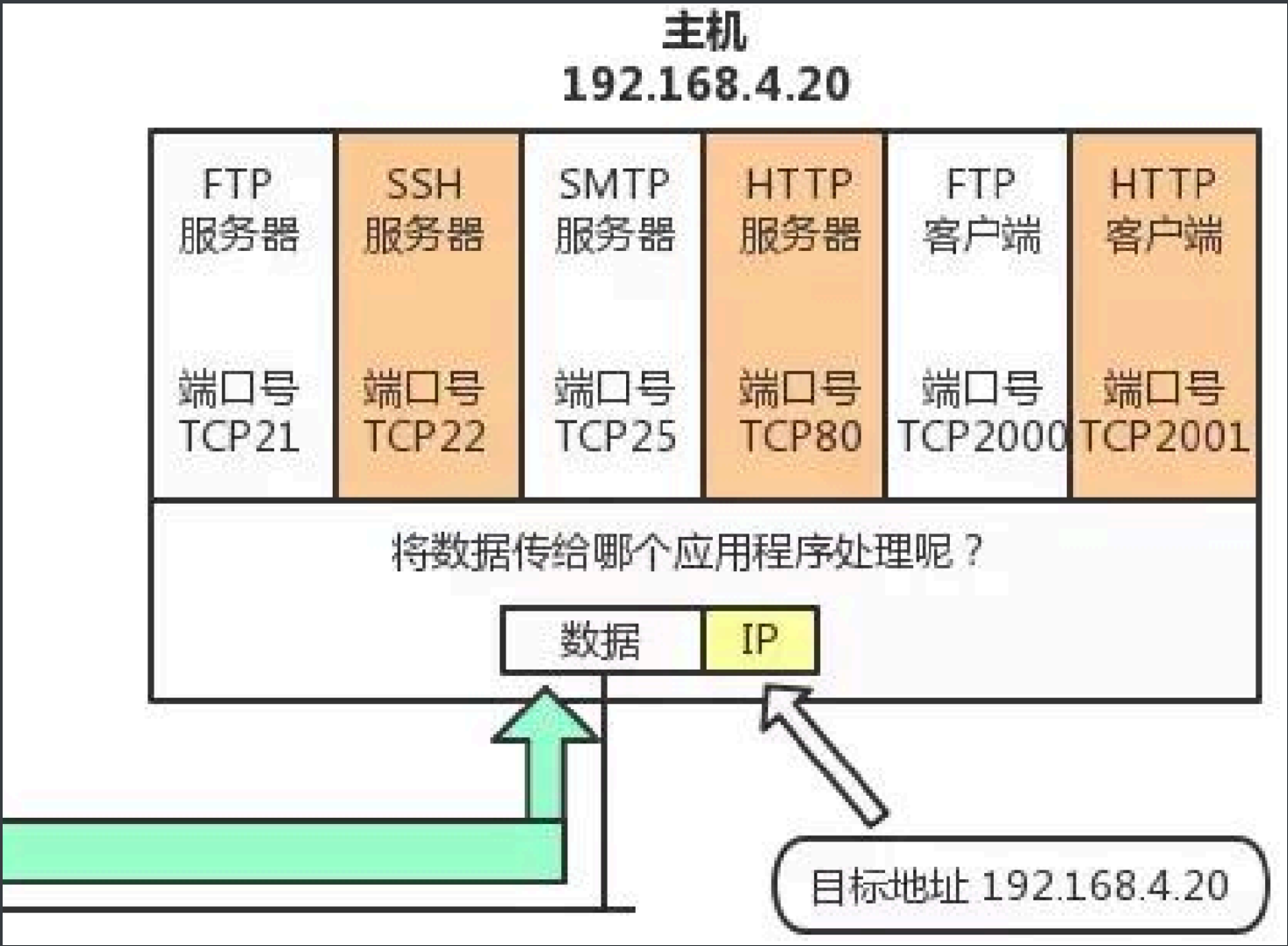
② 通过 `ping` 查看网络连接：

- ① `ping www.baidu.com` 查看是否能上公网
- ② `ping 192.168.1.100` 查看是否和该计算机在同一个局域网
- ③ `ping 127.0.0.1` 查看本机网卡是否可用

端口 port

端口号用来识别计算机中进行通信的应用程序。因此，它也被称为程序地址。

一台计算机上同时可以运行多个程序。传输层协议正是利用这些端口号识别本机中正在进行通信的应用程序，并准确地进行数据传输。



1. 端口分配

端口是虚拟的概念，并不是说在主机上真的有若干个端口。通过端口，可以在一个主机上运行多个网络应用程序。端口的表示是一个16位的二进制整数，对应十进制的 0-65535。操作系统中一共提供了 0~65535 可用端口范围。

2. 公认端口

从0到1023，它们紧密绑定（binding）于一些服务。通常这些端口的通讯明确表明了某种服务的协议。例如：80端口实际上总是HTTP通讯。

3. 注册端口

从1024到65535。它们松散地绑定于一些服务。也就是说有许多服务绑定于这些端口，这些端口同样用于许多其它目的。例如：许多系统处理动态端口从1024左右开始。

网络通信协议

常用网络协议神图

⑦应用层

POP3 邮局协议版本3 110
SMTP 简单邮件传输协议 25
Telnet 远程控制 23
FTP 文件传输协议 21
HTTP 超文本传输协议 80
IMAP 因特网消息访问 143
HTTP-s HTTP安全协议 443

DNS

53

DHCP 动态主机配置协议 67
TFTP 简单文件传送协议 69
SNMP 简单网络管理协议 161
SLP 服务定位协议
NTP 网络时间协议
RADIUS 远程用户拨号认证
BOOTP 引导程序协议

⑥表示层

⑤会话层

LDAP
轻量级目录访问协议

7号信令系统
SS7

④传输层

DVMRP
距离矢量多播选路协议

传送适配层协议

TCP

UDP

Mobile IP 移动IP

SSL/TLS 安全套接字协议

③网络层

BGP 边界网关协议 179

HSRP 热备份路由协议 1985

RIP/RIPng 距离矢量路由协议 520/521

IGMP 因特网组管理协议 2
ICMP 因特网控制报文协议 1
RSVP 资源预留协议 46
ESP IP安全封装协议 50
AH IP安全认证协议 51
X.25 分组交换网协议 93

IP

OSPF 开放最短路径优先 89
IS-IS 中间系统到中间系统协议 124
EGP 外部网关路由协议 8
VRRP 虚拟路由冗余协议 112
IGRP 动态距离矢量路由协议 9
EIGRP 增强IGRP路由协议 88

路由协议

②链路层

MPLS
多协议标签交换

HDLC 高级链路控制
SDLC 同步链路控制

L2F 第二层转发协议
L2TP 第二层隧道协议
PPTP 点对点隧道协议

VPN协议

ATM
异步传输网

PPP 点对点协议
PPPOE 以太网PPP

LACP 链路汇聚控制协议 802.3ad
ISL 思科交换机链路协议 已淘汰

ARP 地址解析协议
RARP 反向地址解析协议

802.1d 冗余链路STP
802.1w 快速STP
802.1q VLAN
802.1x 认证系统
802.1p Qos流量优先级
802.1g 远程网桥

IEEE 802.1

CSMA/CA

IEEE 802.11

IEEE 802

IEEE 802.3

CSMA/CD

①物理层

接入网
有线: E1/E3, T1/T3
窄带: PSTN, ISDN, X.25
宽带: Ether, ADSL, EPON
传输: SDH, RPR, MSTP
无线: GPRS, 3G, 4G

WLAN标准

802.11b 2.4GHz 11Mbps
802.11a 5.0GHz 54Mbps
802.11g 2.4GHz 54Mbps
802.11n 2.4&5 300~600
802.11ac 5.0GHz 500~1G

IEEE 802.15 (蓝牙技术)
IEEE 802.16 (无线城域网)

MAN标准

IEEE 802.3ab 1000M
1000Base-T 四对UTP

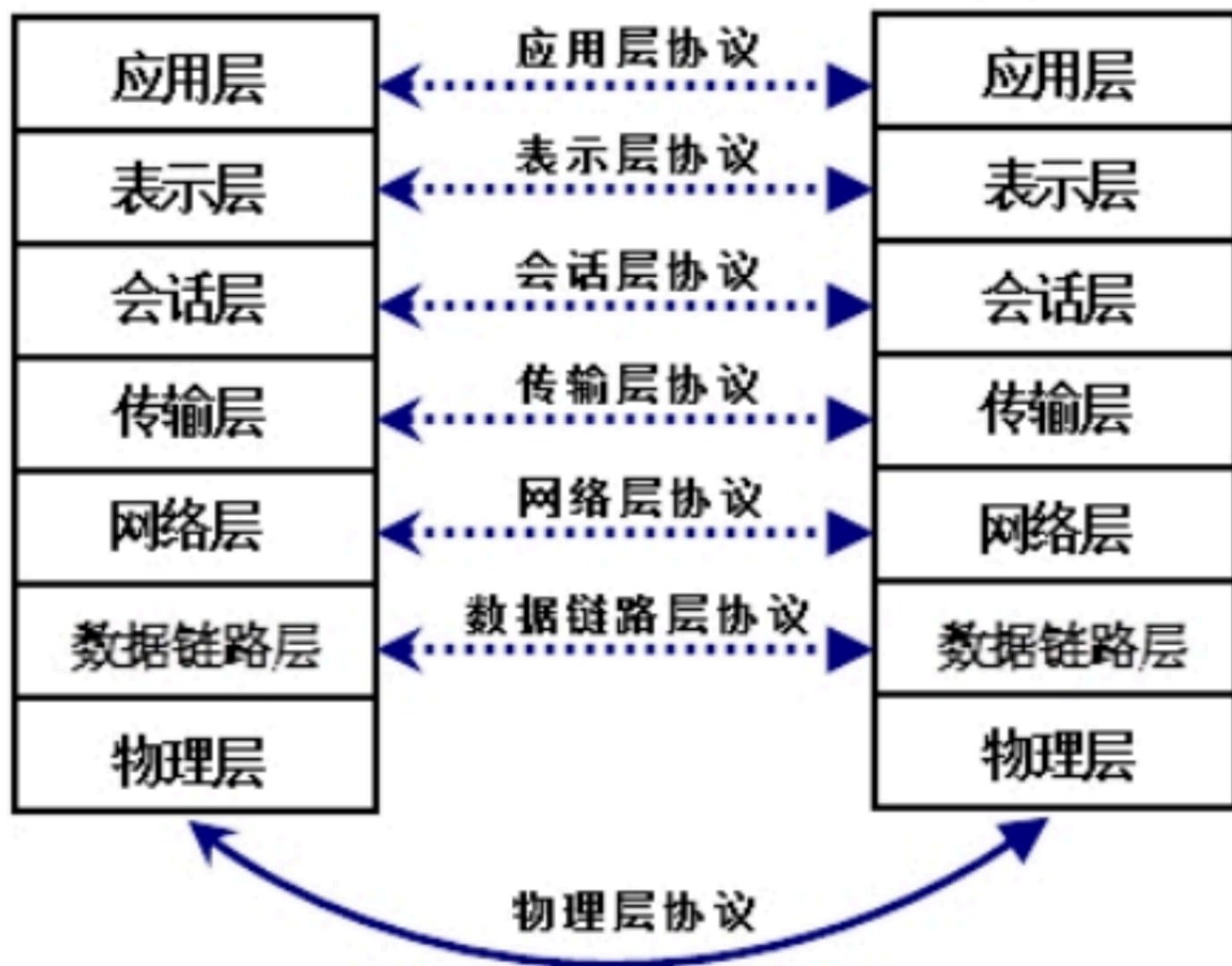
IEEE 802.3ae 10G
10GBase-S 多模光纤 300m
10GBase-L 单模光纤 10Km
10GBase-E 单模光纤 40Km
10GBase-LX4 单模光纤 10Km
IEEE 802.3an 10G
10GBase-T 双绞线 100m

LAN标准

IEEE 802.3z 1000M
1000Base-SX 短波光纤 550m
1000Base-LX 长波光纤 5000m
1000Base-CX 铜对STP 25m

主机A

主机B



ISO参考模型

应用层
表示层
会话层
传输层
网络层
数据链路层
物理层

TCP/IP参考模型

应用层
传输层 (TCP/UDP)
网络层 (IP)
物理 + 数据链路层

TCP和UDP协议区别

	UDP	TCP
是否连接	无连接	面向连接
是否可靠	不可靠传输，不使用流量控制和拥塞控制	可靠传输，使用流量控制和拥塞控制
连接对象个数	支持一对一，一对多，多对一和多对多交互通信	只能是一对一通信
传输方式	面向报文	面向字节流
首部开销	首部开销小，仅8字节	首部最小20字节，最大60字节
适用场景	适用于实时应用（IP电话、视频会议、直播等）	适用于要求可靠传输的应用，例如文件传输

总结:

TCP是**面向连接**的，传输数据安全，稳定，效率相对较低。

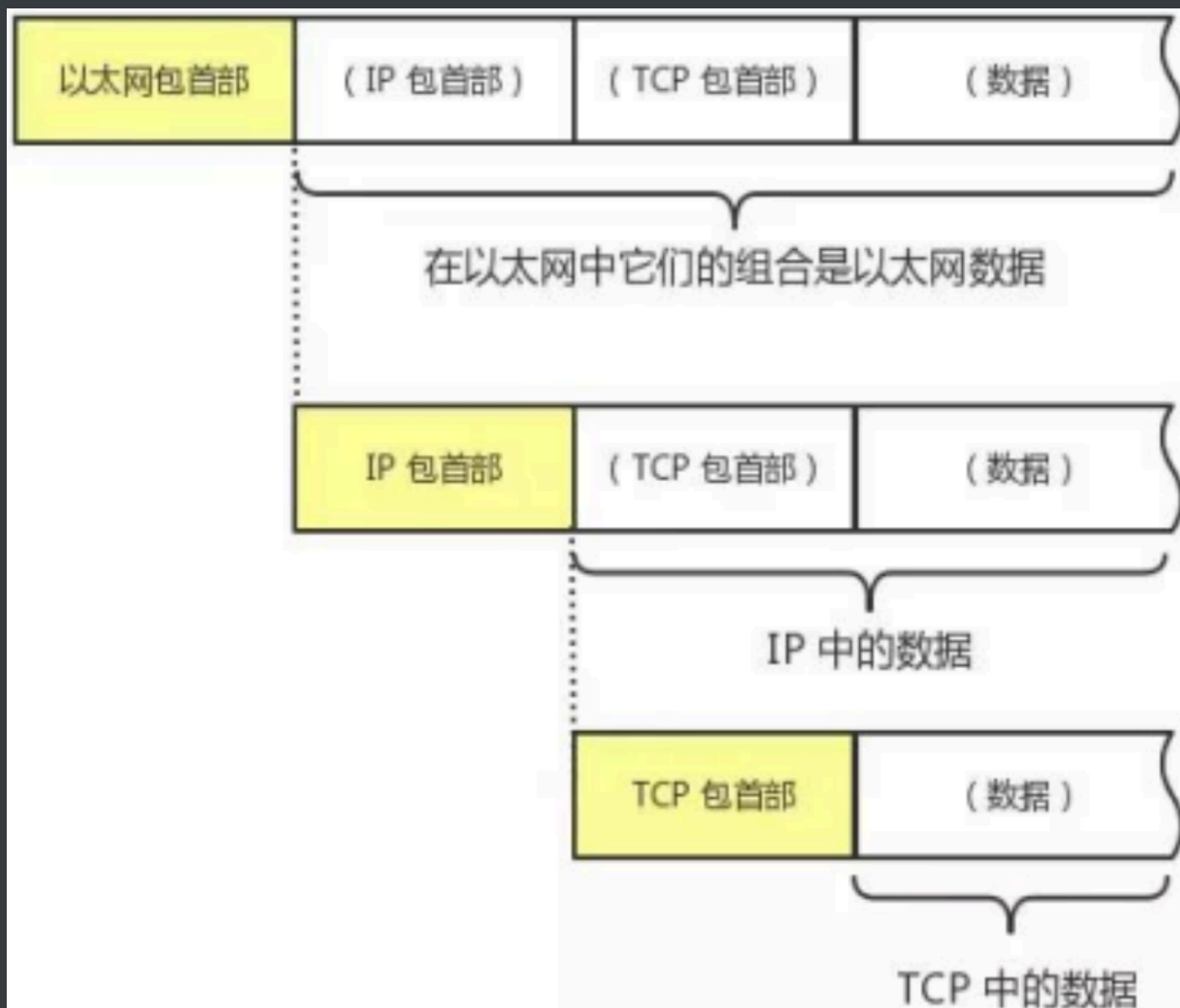
UDP是**面向无连接**的，传输数据不安全，效率较高。

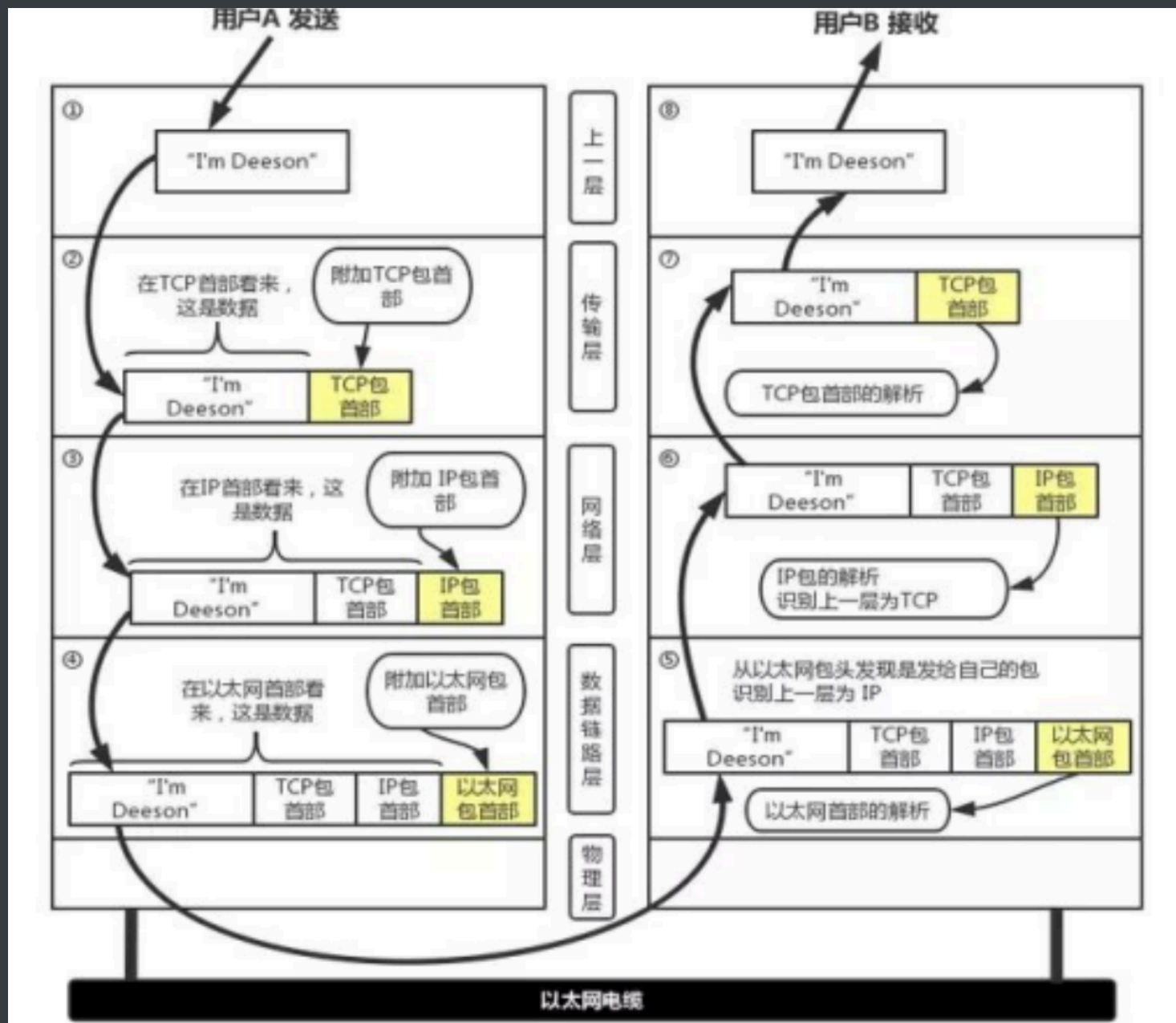
什么是数据包和数据包处理流程

通信传输中的数据单位，一般也称“数据包”。在数据包中包括：包、帧、数据包、段、消息。

网络中传输的数据包由两部分组成：一部分是协议所要用的首部，另一部分是上一层传过来的数据。

首部的结构由协议的具体规范详细定义。在数据包的首部，明确标明了协议应该如何读取数据。反过来说，看到首部，也就能够了解该协议必要的信息以及所要处理的数据。包首部就像协议的脸。



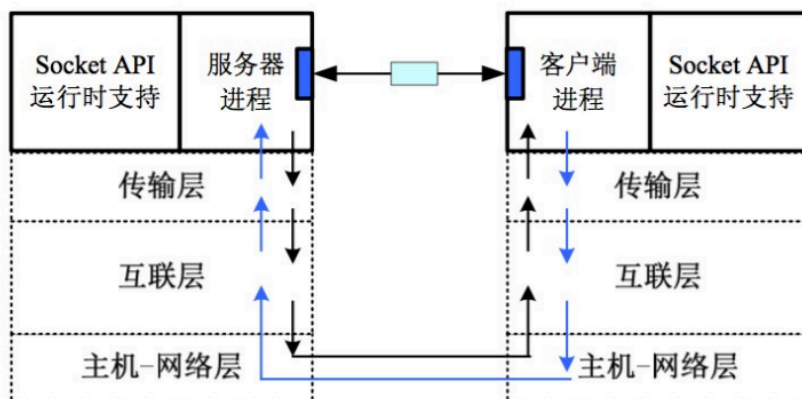
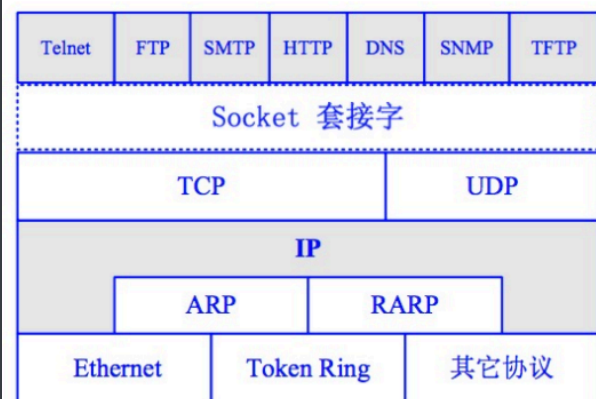
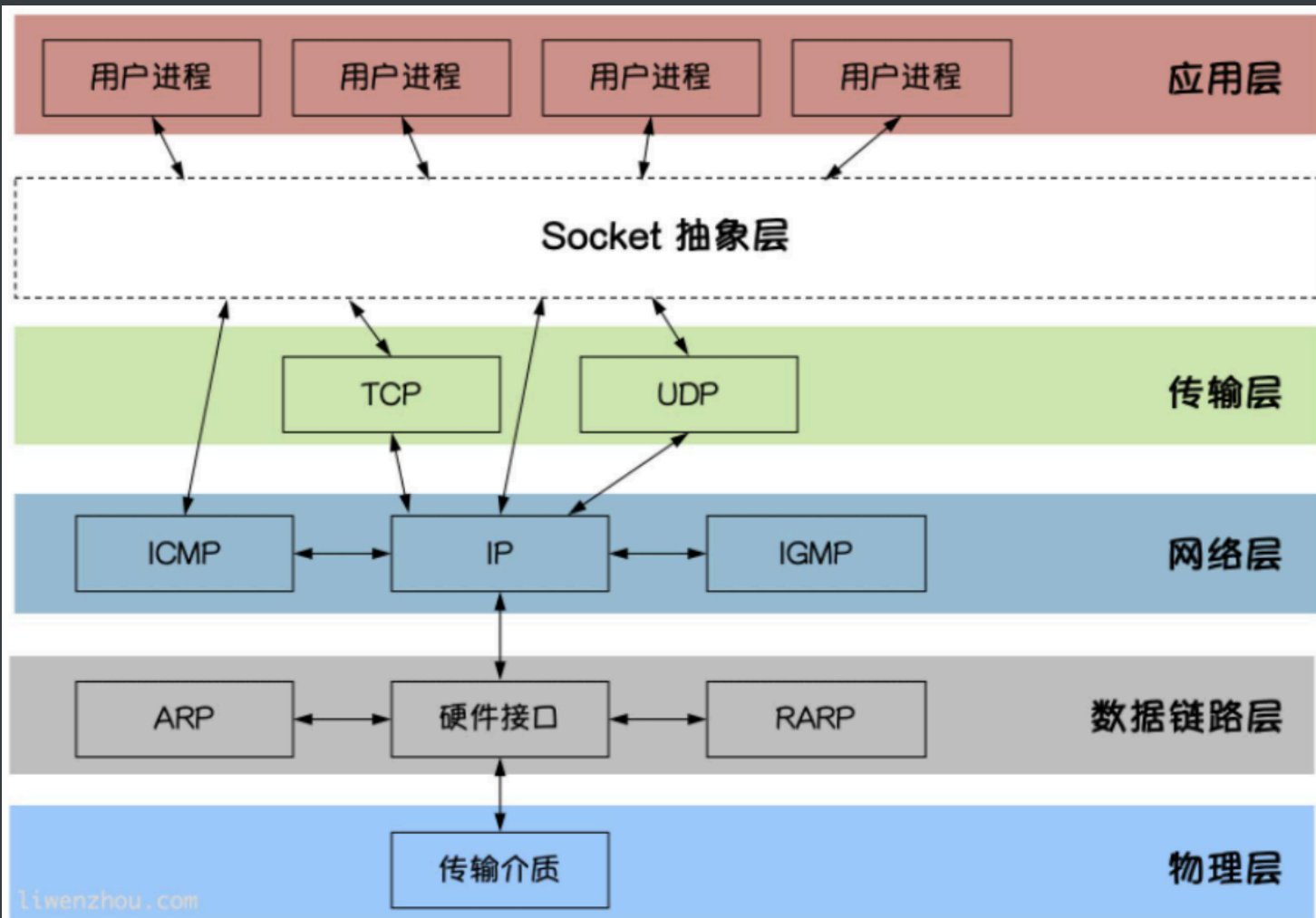


套接字编程实战

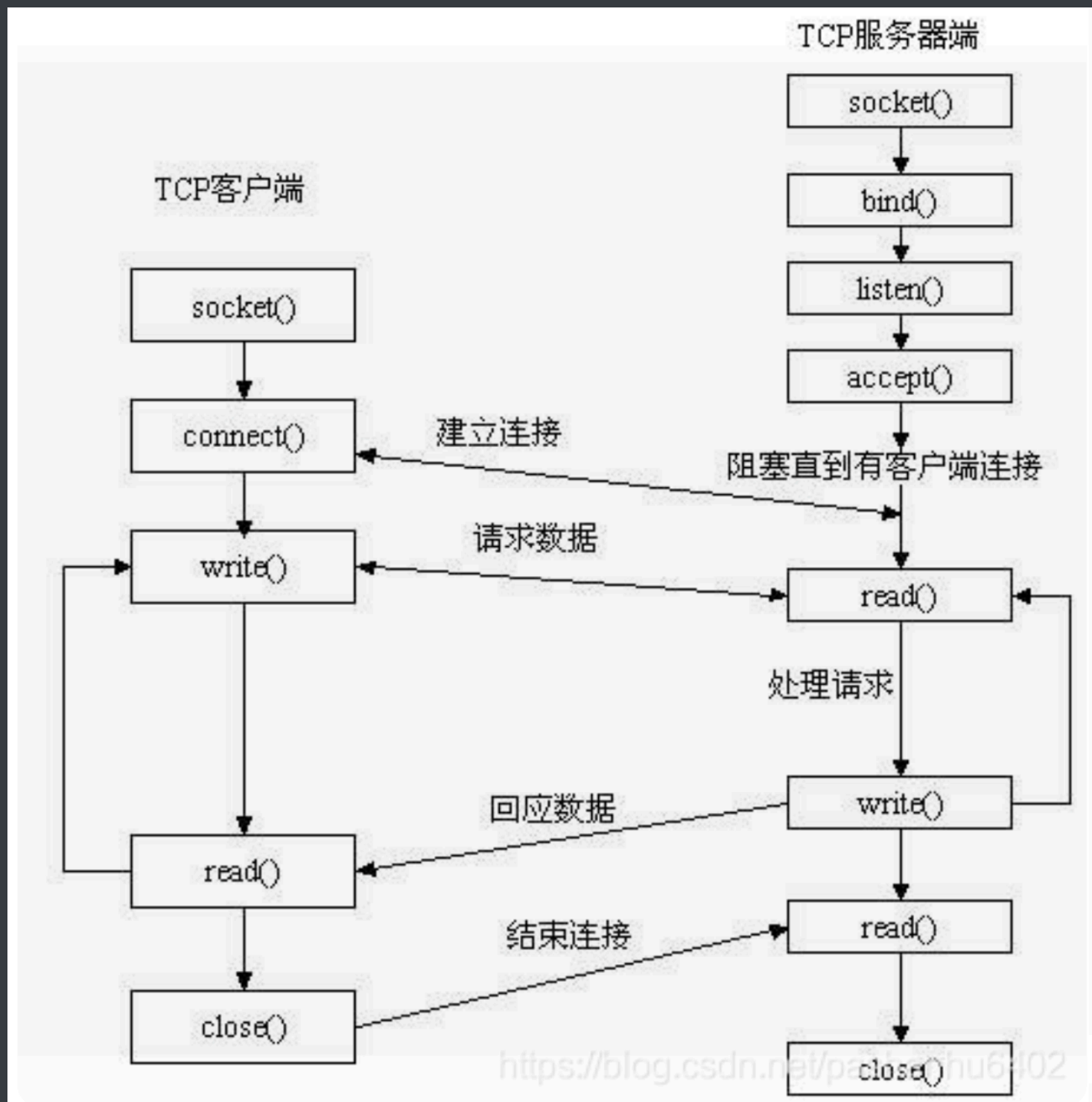
socket编程介绍

TCP协议和UDP协议是传输层的两种协议。

Socket是传输层供给应用层的编程接口，所以Socket编程就分为TCP编程和UDP编程两类。



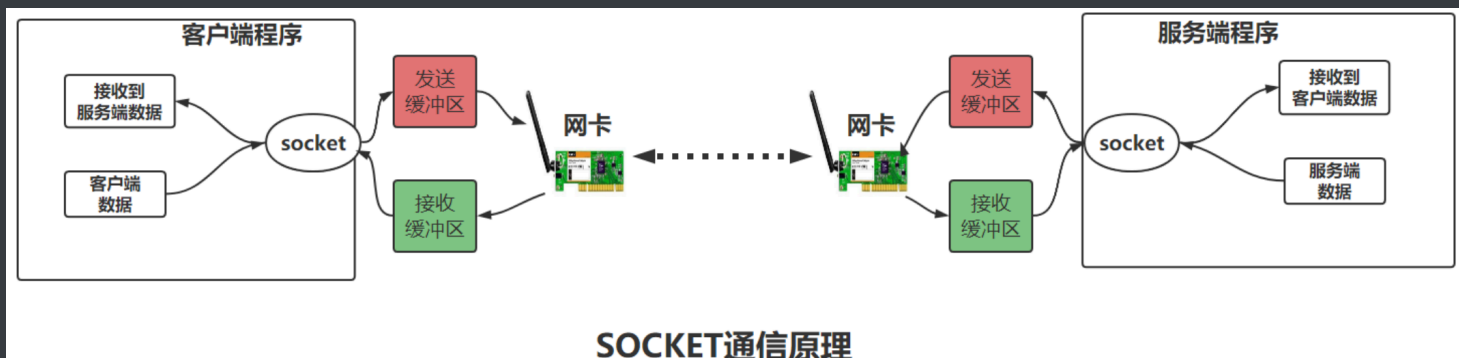
Socket编程封装了常见的TCP、UDP操作，可以实现非常方便的网络编程。



socket() 函数介绍

在Python语言标准库中，通过使用 `socket` 模块提供的`socket`对象，可以在计算机网络中建立可以互相通信的服务器与客户端。

在服务器端需要建立一个`socket`对象，并等待客户端的连接。客户端使用 `socket`对象与服务器端进行连接，一旦连接成功，客户端和服务器端就可以进行通信了。



可以看出socket通讯中，发送和接收数据，都是通过操作系统控制网卡来进行。因此，在使用之后，必须关闭socket。

在 Python 中，通常用一个Socket表示“打开了一个网络连接”，语法格式如下：

```
...  
family : 套接字家族可以使 AF_UNIX 或者 AF_INET;  
        AF 表示 ADDRESS FAMILY 地址族  
        AF_INET (又称 PF_INET) 是 IPv4 网络协议的套接字类型;  
        AF_UNIX 则是 Unix 系统本地通信。  
  
type : 套接字类型可以根据是面向连接的还是非连接分为 SOCK_STREAM 或 SOCK_DGRAM;  
protocol : 一般不填，默认为0。  
...  
socket.socket([family[, type[, protocol]]])
```

Socket主要分为面向连接的Socket和无连接的Socket。

无连接Socket的主要协议是用户数据报协议，也就是常说的UDP，UDP Socket的名字是 SOCK_DGRAM。

创建套接字UDP/IP套接字，可以调用 `socket.socket()`。

示例代码如下：`udpSocket=socket.socket (AF_INET,SOCK_DGRAM)`

socket对象的内置函数和属性

在Python语言中socket对象中，提供如表所示的内置函数。

函数	功能
服务器端套接字函数	
s.bind()	绑定地址（host,port）到套接字， 在AF_INET下,以元组（host,port）的形式表示地址。
s.listen()	开始TCP监听。backlog指定在拒绝连接之前，操作系统可以挂起的最大连接数量。该值至少为1，大部分应用程序设为5就可以了。
s.accept()	被动接受TCP客户端连接,(阻塞式)等待连接的到来
客户端套接字	
s.connect()	主动初始化TCP服务器连接，。一般address的格式为元组（hostname,port），如果连接出错，返回socket.error错误。
s.connect_ex()	connect()函数的扩展版本,出错时返回出错码,而不是抛出异常

公共用途的套接字函数	
<code>s.recv()</code>	接收TCP数据，数据以字符串形式返回，bufsize指定要接收的最大数据量。flag提供有关消息的其他信息，通常可以忽略。
<code>s.send()</code>	发送TCP数据，将string中的数据发送到连接的套接字。返回值是要发送的字节数量，该数量可能小于string的字节大小。
<code>s.sendall()</code>	完整发送TCP数据，完整发送TCP数据。将string中的数据发送到连接的套接字，但在返回之前会尝试发送所有数据。成功返回None，失败则抛出异常。
<code>s.recvfrom()</code>	接收UDP数据，与recv()类似，但返回值是（data,address）。其中data是包含接收数据的字符串，address是发送数据的套接字地址。
<code>s.sendto()</code>	发送UDP数据，将数据发送到套接字，address是形式为（ipaddr, port）的元组，指定远程地址。返回值是发送的字节数。
<code>s.close()</code>	关闭套接字
<code>s.getpeername()</code>	返回连接套接字的远程地址。返回值通常是元组（ipaddr,port）。
<code>s.getsockname()</code>	返回套接字自己的地址。通常是一个元组(ipaddr,port)
<code>s.setsockopt(level,optname,value)</code>	设置给定套接字选项的值。
<code>s.getsockopt(level,optname[.buflen])</code>	返回套接字选项的值。

<code>s.settimeout(timeout)</code>	设置套接字操作的超时期，timeout是一个浮点数，单位是秒。值为None表示没有超时期。一般，超时期应该在刚创建套接字时设置，因为它们可能用于连接的操作（如connect()）
<code>s.gettimeout()</code>	返回当前超时期的值，单位是秒，如果没有设置超时期，则返回None。
<code>s.fileno()</code>	返回套接字的文件描述符。
<code>s.setblocking(flag)</code>	如果flag为0，则将套接字设为非阻塞模式，否则将套接字设为阻塞模式（默认值）。非阻塞模式下，如果调用recv()没有发现任何数据，或send()调用无法立即发送数据，那么将引起socket.error异常。
<code>s.makefile()</code>	创建一个与该套接字相关连的文件

UDP 编程

UDP编程介绍

UDP协议时，不需要建立连接，只需要知道对方的IP地址和端口号，就可以直接发数据包。但是，能不能到达就不知道了。虽然用 UDP传输数据不可靠，但它的优点是比 TCP 协议速度快，对于不要求可靠到达的数据，就可以使用UDP协议。

创建Socket时， `SOCK_DGRAM` 指定了这个Socket的类型是UDP。绑定端口和TCP一样，但是不需要调用 `listen()` 方法，而是直接接收来自任何客户端的数据。 `recvfrom()` 方法返回数据和客户端的地址与端口，这样，服务器收到数据后，直接调用 `sendto()` 就可以把数据用UDP发给客户端。

UDP编程实现

```
import socket

def main():
    # 1、创建 UDP 套接字
    # AF_INET: 使用 IPv4 地址族
    # SOCK_DGRAM: 使用 UDP 协议 (数据报套接字, 无连接、不可靠但高效)
    udp_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    # 2、绑定接收信息端口
    # '0.0.0.0': 监听所有可用的网络接口 (包括 127.0.0.1 和局域网 IP)
    # 9999: 监听的端口号, 发送方需要向此端口发送数据
    local_addr = ('0.0.0.0', 9999)
    udp_socket.bind(local_addr)
    print(f'UDP 服务器已启动, 监听地址: {local_addr}')

    # 3、接收数据 (阻塞等待, 直到收到数据)
    # 1024: 本次接收的最大字节数, 如果数据超过 1024 字节, 多余部分会被丢弃
    # recvfrom() 返回一个元组: (数据, (客户端IP, 客户端端口))
    print('等待接收数据...')
    recv_data, client_addr = udp_socket.recvfrom(1024)

    # 4、打印接收到的数据
    # recv_data: 接收到的字节数据 (bytes 类型)
    # client_addr: 发送方的地址信息, 格式为 (IP, 端口)
    print(f'原始数据: {recv_data}')
    print(f'解码后内容: {recv_data.decode("utf-8")}')
    print(f'来自客户端: {client_addr[0]}:{client_addr[1]}')

    # 5、关闭套接字, 释放系统资源
    udp_socket.close()

if __name__ == '__main__':
    main()
```

接收端

D:\proj\threads\.venv\Scripts\python.exe D:\proj\threads\src\network\recv.py
等待接收数据!
(b'\xe4\xbd\xa0\xe5\xa5\xbd', ('192.168.31.106', 49265))
收到远程信息: 你好, from ('192.168.31.106', 49265)

进程已结束, 退出代码为 0

send.py

src > network > send.py > ...
1 import socket
2
3 s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
4
5 data = '你好'
6 s.sendto(data.encode('utf-8'), ('192.168.31.106', 9999))
7
8 s.close()

问题 输出 终端

dataset git:(master) X .venv/bin/python ./src/network/send.py

dataset git:(master) X .venv/bin/python ./src/network/send.py

dataset git:(master) X .venv/bin/python ./src/network/send.py

dataset git:(master) X .venv/bin/python ./src/network/send.py

dataset git:(master) X .venv/bin/python ./src/network/send.py

持续通信

```
import socket

def main():
    1 个用法
    udp_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    local_addr = ('0.0.0.0', 9999)
    udp_socket.bind(local_addr)
    print(f'UDP 服务器已启动, 监听地址: {local_addr}')

    while True:
        print('等待接收数据...')
        recv_data, client_addr = udp_socket.recvfrom(1024)

        data = recv_data.decode('utf-8')
        print(f'来自客户端: {data}')

        if data == 'bye':
            print('退出接收程序')
            break

    udp_socket.close()

if __name__ == '__main__':
    main()
```

↑

↓

↔

↕

🖨

🗑

D:\proj\threads\.venv\Scripts\python.exe D:\proj\threads\src\network\send.py
UDP 服务器已启动, 监听地址: ('0.0.0.0', 9999)
等待接收数据...
来自客户端: 你好
等待接收数据...
来自客户端: 吃饭了吗
等待接收数据...
来自客户端: bye
退出接收程序

send.py U X

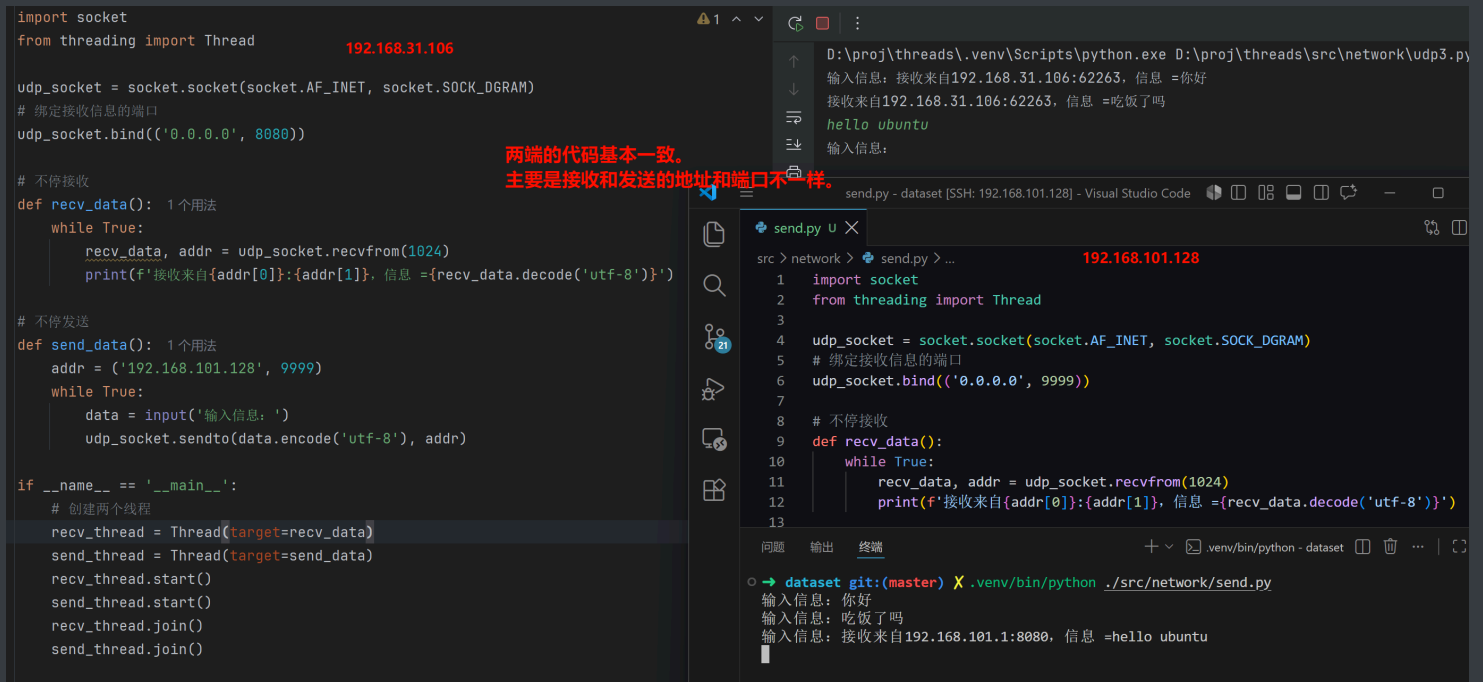
src > network > send.py > ...
1 import socket
2
3 s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
4
5 addr = ('192.168.31.106', 9999)
6
7 while True:
8 data = input('请输入要发送的数据: ')
9 s.sendto(data.encode('utf-8'), addr)
10 if data == 'bye':
11 print('发送数据结束')
12 break
13
14 s.close()

问题 输出 终端

➔ dataset git:(master) X .venv/bin/python ./src/network/send.py
请输入要发送的数据: 你好
请输入要发送的数据: 吃饭了吗
请输入要发送的数据: bye
发送数据结束
➔ dataset git:(master) X

结合多线程实现UDP双向自由通信

UDP 不同于 TCP，不存在请求连接和受理过程，因此在某种意义上无法明确区分服务器端和客户端，只是因为其提供服务而称为服务器端。

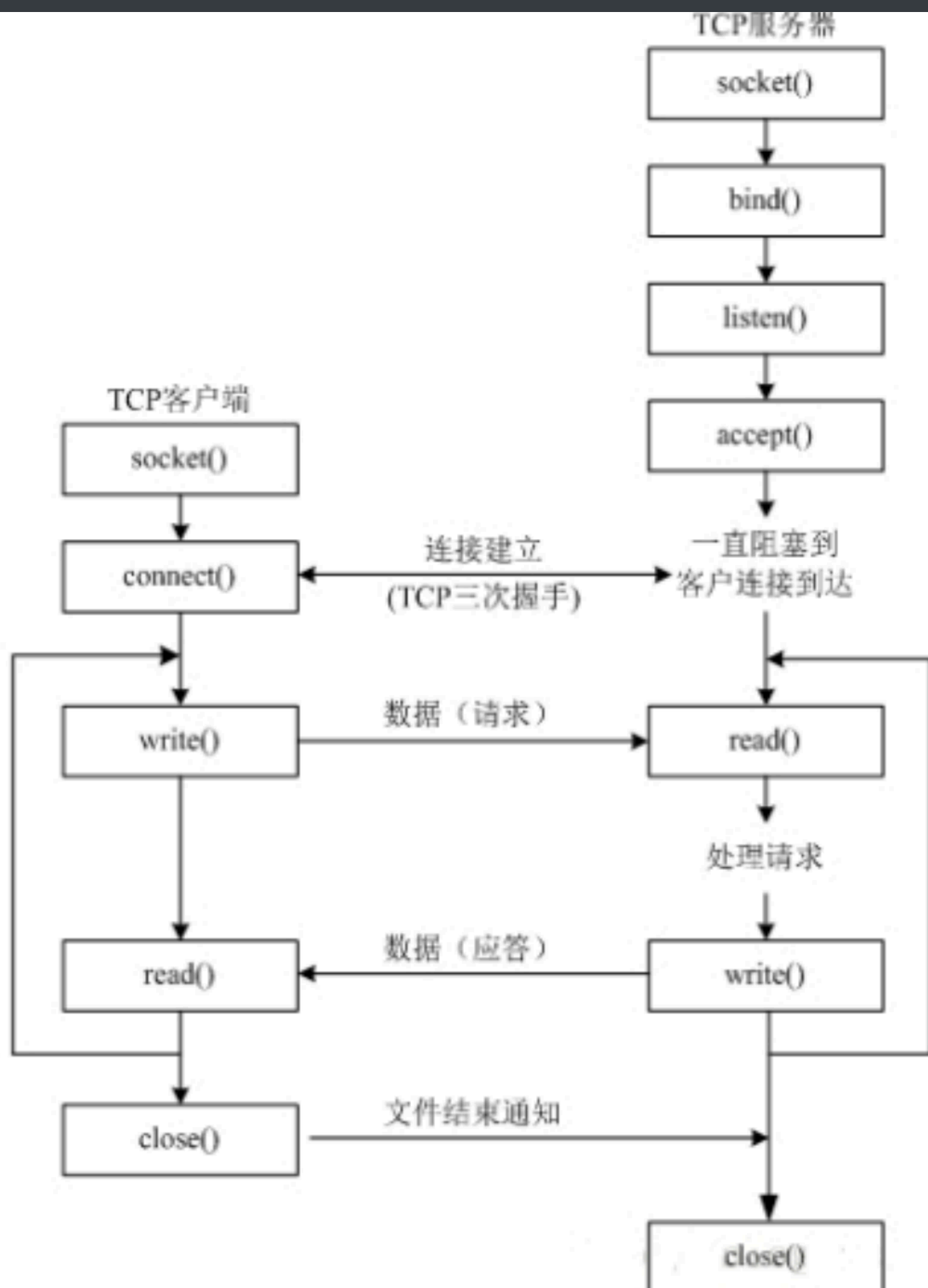


TCP 编程

TCP 编程介绍

面向连接的Socket使用的协议是TCP协议。TCP的Socket名称是 `SOCK_STREAM` 。

创建套接字TCP套接字，可以调用 `socket.socket()` 。 `tcpSocket=socket.socket(AF_INET, SOCK_STREAM)`



TCP编程的实现

在Python语言中创建Socket服务端程序，需要使用socket模块中的 `socket` 类。创建Socket服务器程序的步骤如下：

- 1. 创建 `socket` 对象
- 2. 绑定端口号
- 3. 监听端口号
- 4. 等待客户端 `socket` 的连接
- 5. 读取客户端发送过来的数据
- 6. 向客户端发送数据
- 7. 关闭客户端 `socket` 连接
- 8. 关闭服务端 `socket` 连接

```
# 1、创建 TCP 套接字
# AF_INET: 使用 IPv4 地址族
# SOCK_STREAM: 使用 TCP 协议（流式套接字，面向连接、可靠传输）
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 2、绑定服务端地址和端口
# '0.0.0.0': 监听所有可用的网络接口（包括 127.0.0.1 和局域网 IP）
# 8080: 监听的端口号，客户端需要向此端口发起连接
server_socket.bind(('0.0.0.0', 8080))

# 3、将套接字转为被动监听模式
# 5: 最大等待连接数（超过此数量的连接请求将被拒绝）
# 设置后，服务器正式进入"等待客户端连接"状态
server_socket.listen(5)
print('TCP 服务器已启动，等待客户端连接...')

# 4、接受客户端连接（阻塞等待，直到有客户端连接）
# accept() 返回一个元组：（新的客户端套接字，客户端地址信息）
# 新套接字用于和该客户端通信，原 server_socket 继续监听其他连接
cli_socket, cli_addr = server_socket.accept()
print(f'客户端 {cli_addr} 已连接！')

# 5、接收客户端发送的数据
# 1024: 本次接收的最大字节数，如果数据超过 1024 字节，多余部分会缓存在内核中
# recv() 用于 TCP 接收数据（不需要指定地址，因为是面向连接的）
# 注意：recv() 是阻塞的，如果客户端关闭连接，会返回空字节串 b''
recv_data = cli_socket.recv(1024)

# 6、打印接收到的数据
# decode('gbk'): 使用 GBK 编码解码（Windows 中文系统常用编码）
# 如果发送方使用的是 UTF-8，应改为 decode('utf-8')
print(f'收到信息：{recv_data.decode('gbk')}，来自：{cli_addr}')
# 7、关闭客户端套接字，释放与该客户端的连接资源
cli_socket.close()
# 8、关闭服务端套接字，释放端口，停止监听
server_socket.close()
```

服务端

```
D:\proj\threads\.venv\Scripts\python.exe D:\proj\threads\src\network\tcp1.py
TCP 服务器已启动，等待客户端连接...
客户端 ('192.168.31.106', 62546) 已连接!
收到信息：你好，服务端，来自：('192.168.31.106', 62546)

进程已结束，退出代码为 0
```

send.py - dataset [SSH: 192.168.101.128] - Visual Studio Code

send.py U X

src > network > send.py > ...

2

3 cli_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

4 cli_socket.connect(('192.168.31.106', 8080))

5

6

7

8 注意：

9 1. tcp客户端已经链接好了服务器，所以在以后的数据发送中，不需要填写对方的ip和port

10 2. udp在发送数据的时候，因为没有之前的链接，所以需要在每次的发送中，都要填写接收方的ip和port

11

12 cli_socket.send("你好，服务端".encode('gbk'))

cli_socket.close()

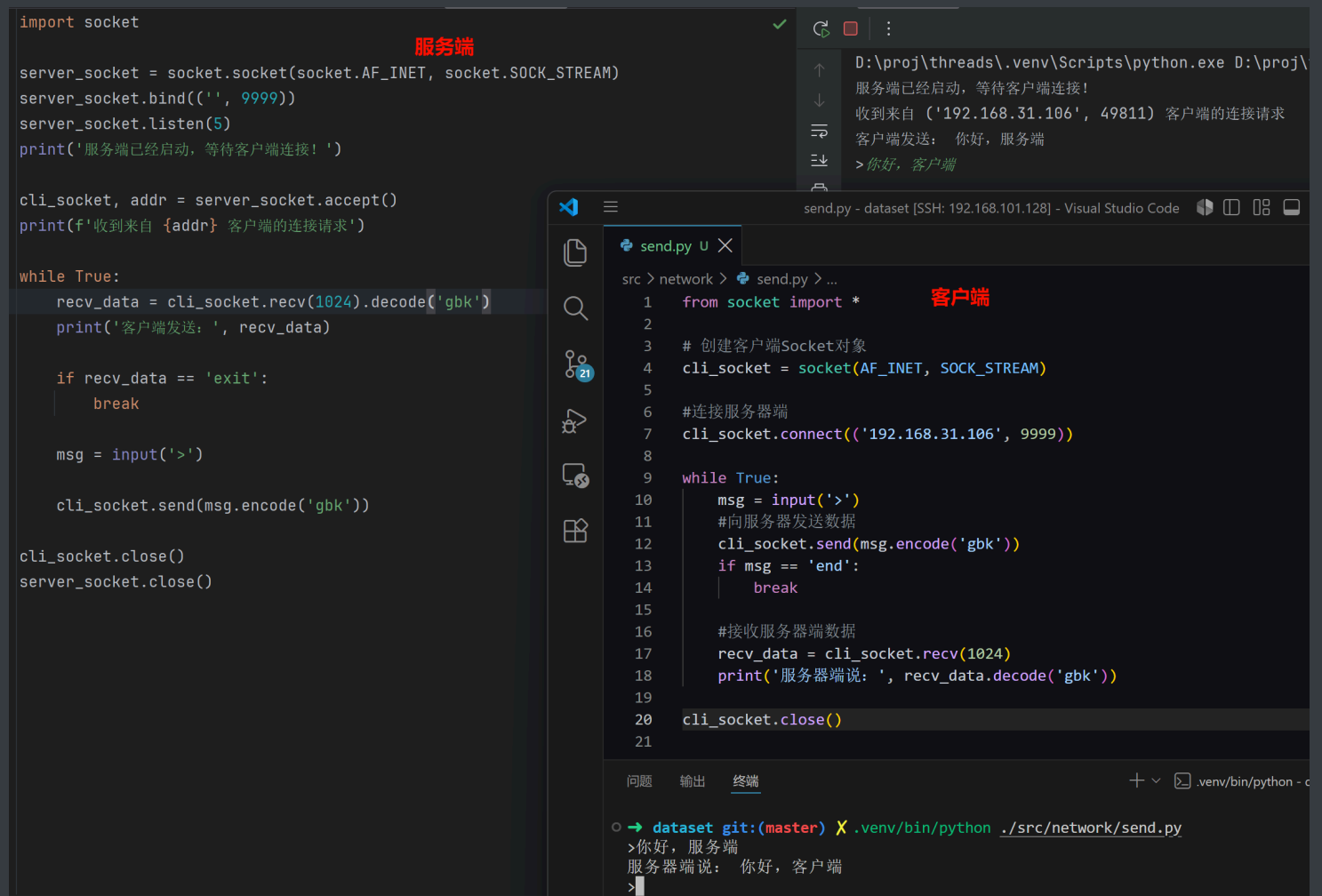
问题 输出 终端

zsh - dataset

dataset git:(master) X .venv/bin/python ./src/network/send.py

dataset git:(master) X

TCP双向持续通信



```
import socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(('', 9999))
server_socket.listen(5)
print('服务端已经启动, 等待客户端连接!')

cli_socket, addr = server_socket.accept()
print(f'收到来自 {addr} 客户端的连接请求')

while True:
    recv_data = cli_socket.recv(1024).decode('gbk')
    print('客户端发送: ', recv_data)

    if recv_data == 'exit':
        break

    msg = input('>')

    cli_socket.send(msg.encode('gbk'))

cli_socket.close()
server_socket.close()
```

服务端

```
from socket import *

# 创建客户端Socket对象
cli_socket = socket(AF_INET, SOCK_STREAM)

#连接服务器端
cli_socket.connect(('192.168.31.106', 9999))

while True:
    msg = input('>')
    #向服务器发送数据
    cli_socket.send(msg.encode('gbk'))
    if msg == 'end':
        break

    #接收服务器端数据
    recv_data = cli_socket.recv(1024)
    print('服务器端说: ', recv_data.decode('gbk'))

cli_socket.close()
```

客户端

终端输出:

```
dataset git:(master) X .venv/bin/python ./src/network/send.py
>你好, 服务端
服务器端说:  你好, 客户端
>
```

多线程实现TCP双线传送（自由聊天）

```
import socket
from threading import Thread

def recv_msg(cli):
    """接收服务器消息的线程"""
    while True:
        data = cli.recv(1024).decode('gbk')
        if not data:
            print('\n服务器已断开连接')
            break
        elif data == 'exit':
            print('\n服务器请求退出')
            break
```

```

        print(f'\n服务器>> {data}')
        print('请输入消息 > ', end='', flush=True)

def send_msg(cli):
    """发送消息给服务器的线程"""
    while True:
        msg = input('请输入消息 > ')
        cli.send(msg.encode('gbk'))
        if msg == 'exit':
            break

if __name__ == '__main__':
    # 连接服务器
    cli = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    cli.connect(('192.168.31.106', 9999))
    print('已连接到服务器, 输入 exit 退出')

    # 创建收发线程
    recv_thread = Thread(target=recv_msg, args=(cli,))
    send_thread = Thread(target=send_msg, args=(cli,))
    recv_thread.daemon = True
    send_thread.daemon = True

    recv_thread.start()
    send_thread.start()

    # 等待发送线程结束
    send_thread.join()
    cli.close()
    print('客户端已关闭')

```

```

import socket
from threading import Thread

def recv_msg(cli):
    """接收服务器消息的线程"""
    while True:
        data = cli.recv(1024).decode('gbk')

```

```

        if not data:
            print('\n服务器已断开连接')
            break
        elif data == 'exit':
            print('\n服务器请求退出')
            break

        print(f'\n服务器>> {data}')
        print('请输入消息 > ', end='', flush=True)

def send_msg(cli):
    """发送消息给服务器的线程"""
    while True:
        msg = input('请输入消息 > ')
        cli.send(msg.encode('gbk'))
        if msg == 'exit':
            break

if __name__ == '__main__':
    # 连接服务器
    cli = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    cli.connect(('192.168.31.106', 9999))
    print('已连接到服务器, 输入 exit 退出')

    # 创建收发线程
    recv_thread = Thread(target=recv_msg, args=(cli,))
    send_thread = Thread(target=send_msg, args=(cli,))
    recv_thread.daemon = True
    send_thread.daemon = True

    recv_thread.start()
    send_thread.start()

    # 等待发送线程结束
    send_thread.join()
    cli.close()
    print('客户端已关闭')

```

```
D:\proj\threads\.venv\Scripts\python.exe D:\proj\threads\s
```

聊天服务已启动，监听端口：9999

等待客户端连接...

新客户端连接：192.168.31.106:58300

输入信息：192.168.31.106:58300>> 你好

192.168.31.106:58300>> 我不好

新客户端连接：192.168.31.106:56236

输入信息：192.168.31.106:56236>> 你好呀

192.168.31.106:56236>> 不好呢

192.168.31.106:56236>> 嘿嘿

192.168.31.106:58300>> java